SAVEETHA SCHOOL OF ENGINEERING
SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES

C PROGRAMMING ASSIGNMENT

Student Submission Template

Student Name	Tharun
Register Number	1924260280
Course Code & Name	CSA02 — C Programming
Slot	B
Assignment / Task Title	Design, Implement and Validate a Reliable Billing Program for a Rural Electricity Co-operative
GitHub Repository URL	https://github.com/<your-username>/CSA02-1924260280-Assignment1
Submission Date	10 November 2026

Instructions to Students

1. Complete every section for each assigned task.
2. Write your own explanation, algorithm/pseudocode, and program.
3. Run the program and insert a clear screenshot of the output.
4. Include test cases and a short analysis where applicable.
5. If code is maintained on GitHub, provide the correct repository link.

TASK 1 — Understand the Problem and Identify the Data

Task / Problem Statement

The Grama Jyothi co-operative bills about 4,000 households a month from the previous and current meter readings. The old spreadsheet gave bills that were off by a few paise, gave negative bills when a meter rolled over from 65535 to 0, and applied a tariff revision only partly. Before writing the billing program, every item of data it must store and compute has to be identified and justified: the inputs, the outputs, the constants, the values that can be negative, fractional or very large, the rules every correct bill must obey, and the assumptions made.

Aim

To analyse the billing problem and list, with a reason for each, all the data items, constants, rules and assumptions needed by a correct billing program.

Concept / Theory

Inputs and outputs

Data item	Kind	Justification
Previous reading	Input	Whole number read from a 16-bit register (0 to 65535); never negative or fractional.
Current reading	Input	Same register; may be smaller than the previous reading after a roll-over.
Household number	Input	Only an identifier, never used in arithmetic; the district has 800,000 houses, so it needs more than 16 bits.
Category	Input	One letter (D = domestic, C = commercial); a single character is enough.
Units	Output	Current minus previous, with roll-over handled; whole and never negative.
Energy charge	Output	Sum of slab charges; stored as an integer number of paise so that no rounding occurs.
Fixed charge	Output	Rs 50 (5000 paise) on every bill.
Total	Output	Fixed charge + energy charge, in paise.

What should be constants

These values never change during a run, so they are named constants and not typed in several places (the cause of the partial tariff revision): slab limits 100 and 200 units, rates 0, 225 and 450 paise per unit, the fixed charge 5000 paise, the register maximum 65535, 100 paise per rupee, and the largest plausible monthly consumption used to tell a roll-over from a mistake.

Negative, fractional and very large values

- Negative: a raw reading can never be negative, but the difference current − previous is negative after a roll-over. This was the spreadsheet bug, so the subtraction must use wrap-around (unsigned) arithmetic.
- Fractional: a rate such as Rs 2.25 is fractional in rupees but a whole number (225) in paise, so all money is kept in integer paise.
- Very large: one bill is small, but the total of a district (800,000 bills) is about 6.5 × 10^10 paise, which does not fit in a 32-bit integer.

Rules every correct bill must satisfy

- The total is never negative. The total equals the fixed charge plus the energy charge.
- Units are never negative. A bill never decreases when the units increase. 0 to 100 units cost only the fixed charge.

Assumptions

One reading per household per month; at most one roll-over between two readings; readings are whole units; a roll-over is accepted only if the wrapped consumption is at most 5000 units; the category does not change the tariff in this assignment; the tariff is Rs 50 fixed, first 100 units free, next 100 at Rs 2.25 and the rest at Rs 4.50.

Procedure / Algorithm

1. List every input the meter reader supplies and every value the office must compute.
2. Decide the kind of each item: whole number, character, or money.
3. Mark the values that never change and give them constant names.
4. Write the rules a correct bill must satisfy and write down the assumptions.
5. Write a C program that stores one bill in a structure, computes it, and checks the three rules.
6. Measure the size of each item and compare integer readings with text readings.

Program / Source Code

data_items.c

```
/* Task 1 - Identify the data items and check the rules every bill must satisfy */
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <time.h>

#define FIXED_CHARGE_PAISE 5000L
#define SLAB1_UNITS 100
#define SLAB2_UNITS 200
#define RATE2_PAISE 225
#define RATE3_PAISE 450

typedef unsigned short meter_t;          /* 16-bit register, 0..65535 */

typedef struct {
    unsigned int household_no;           /* identifier, never negative   */
    char         category;               /* 'D' domestic, 'C' commercial */
    meter_t      previous, current;      /* raw register values          */
    unsigned int units;                  /* consumption, never negative  */
    long         energy_paise;           /* integer paise: no rounding   */
    long         fixed_paise;
    long         total_paise;
} bill_t;

static void make_bill(bill_t *b)
{
    int u = (int)b->units;
    b->fixed_paise  = FIXED_CHARGE_PAISE;
    b->energy_paise = 0;
    if (u > SLAB2_UNITS) { b->energy_paise += (long)(u - SLAB2_UNITS) * RATE3_PAISE; u = SLAB2_UNITS; }
    if (u > SLAB1_UNITS)   b->energy_paise += (long)(u - SLAB1_UNITS) * RATE2_PAISE;
    b->total_paise = b->fixed_paise + b->energy_paise;
}

int main(void)
{
    bill_t b = {1042, 'D', 65400, 120, 0, 0, 0, 0};
    b.units = (meter_t)(b.current - b.previous);        /* roll-over safe */
    make_bill(&b);

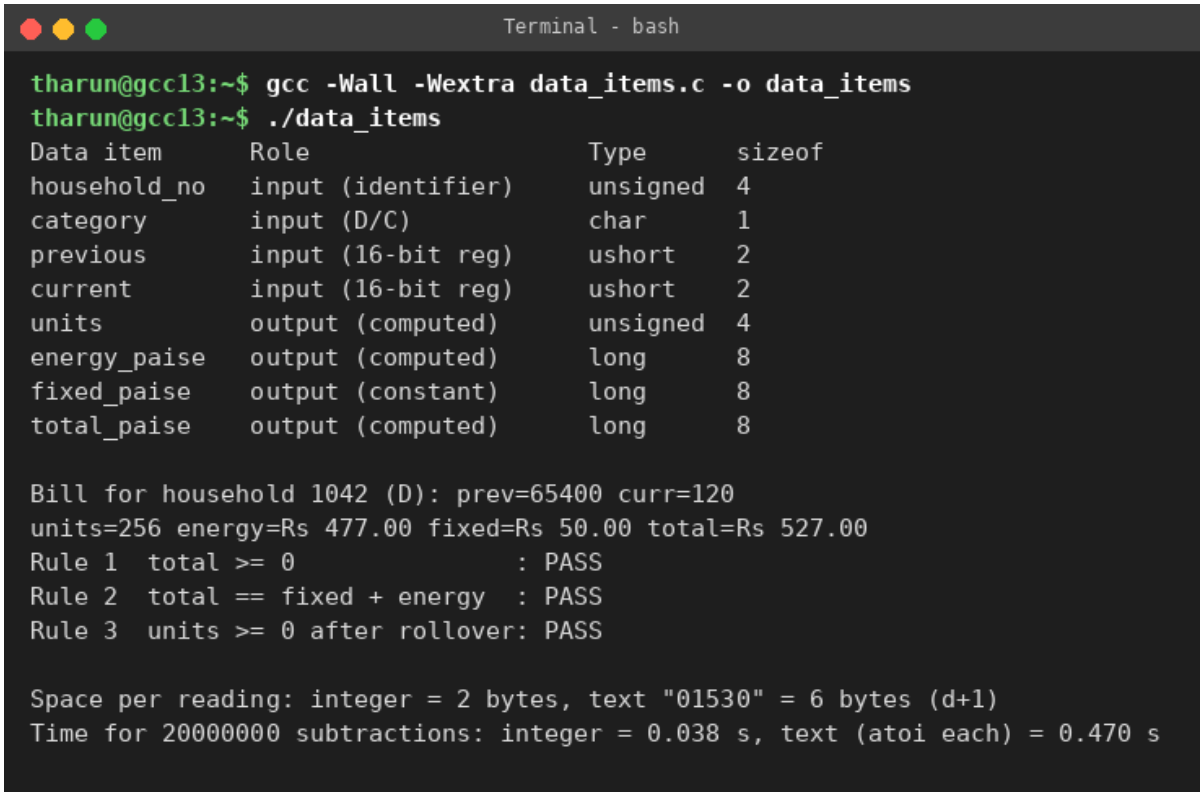
    printf("%-14s %-22s %-9s %s\n", "Data item", "Role", "Type", "sizeof");
    printf("%-14s %-22s %-9s %zu\n", "household_no", "input (identifier)", "unsigned", sizeof
        b.household_no);
    printf("%-14s %-22s %-9s %zu\n", "category",     "input (D/C)",        "char",     sizeof
        b.category);
    printf("%-14s %-22s %-9s %zu\n", "previous",     "input (16-bit reg)", "ushort",   sizeof
        b.previous);
    printf("%-14s %-22s %-9s %zu\n", "current",      "input (16-bit reg)", "ushort",   sizeof
        b.current);
    printf("%-14s %-22s %-9s %zu\n", "units",        "output (computed)",  "unsigned", sizeof b.units);
    printf("%-14s %-22s %-9s %zu\n", "energy_paise", "output (computed)",  "long",     sizeof
        b.energy_paise);
    printf("%-14s %-22s %-9s %zu\n", "fixed_paise",  "output (constant)",  "long",     sizeof
        b.fixed_paise);
    printf("%-14s %-22s %-9s %zu\n", "total_paise",  "output (computed)",  "long",     sizeof
        b.total_paise);

    printf("\nBill for household %u (%c): prev=%u curr=%u\n", b.household_no, b.category,
           (unsigned)b.previous, (unsigned)b.current);
    printf("units=%u energy=Rs %ld.%02ld fixed=Rs %ld.%02ld total=Rs %ld.%02ld\n", b.units,
```

```
           b.energy_paise/100, b.energy_paise%100, b.fixed_paise/100, b.fixed_paise%100,
           b.total_paise/100, b.total_paise%100);
    printf("Rule 1  total >= 0              : %s\n", b.total_paise >= 0 ? "PASS" : "FAIL");
    printf("Rule 2  total == fixed + energy  : %s\n", b.total_paise == b.fixed_paise + b.energy_paise ?
        "PASS" : "FAIL");
    printf("Rule 3  units >= 0 after rollover: %s\n", (int)b.units >= 0 ? "PASS" : "FAIL");

    /* Alternative concept: readings stored as text "01530" instead of integers */
    enum { N = 20000000 };
    char (*ta)[6] = malloc((size_t)N * 6), (*tb)[6] = malloc((size_t)N * 6);
    meter_t *ia = malloc(N * sizeof(meter_t)), *ib = malloc(N * sizeof(meter_t));
    for (int i = 0; i < N; i++) {
        int x = (i * 7919) % 65536, y = (x + 37) % 65536;
        ia[i] = (meter_t)x; ib[i] = (meter_t)y;
        snprintf(ta[i], 6, "%05d", x % 100000); snprintf(tb[i], 6, "%05d", y % 100000);
    }
    volatile long sink = 0; clock_t t0 = clock();
    for (int i = 0; i < N; i++) sink += (meter_t)(ib[i] - ia[i]);
    double t_int = (double)(clock() - t0) / CLOCKS_PER_SEC;
    t0 = clock();
    for (int i = 0; i < N; i++) sink += (meter_t)(atoi(tb[i]) - atoi(ta[i]));
    double t_txt = (double)(clock() - t0) / CLOCKS_PER_SEC;
    printf("\nSpace per reading: integer = %zu bytes, text \"01530\" = 6 bytes (d+1)\n",
        sizeof(meter_t));
    printf("Time for %d subtractions: integer = %.3f s, text (atoi each) = %.3f s\n", N, t_int, t_txt);
    return 0;
}
```

Output Screenshot



Output of data_items.c

Analysis / Discussion

The program shows that household 1042 moved from 65400 to 120, which gives 256 units, an energy charge of Rs 477.00 and a total of Rs 527.00. All three rules passed. Using unsigned 16-bit subtraction made the roll-over case correct without any if-statement.

Alternative Concept and Complexity Analysis

If the readings were stored as text such as "01530", every calculation would first have to convert the text to a number (atoi or strtol), which reads each of the d digits once, so the time is O(d) per reading instead of O(1) for an integer subtraction. A text reading needs d + 1 bytes (6 bytes for 5 digits including the terminating character), while an unsigned short needs a fixed 2 bytes. Text comparison with strcmp is also O(d) and is correct only if all readings have the same number of digits. In my measurement, 20,000,000 subtractions took 0.038 s with integers and 0.470 s with text conversion, about 12 times slower. The text version is also less safe because letters or spaces can enter the data.

Result

The inputs, outputs, constants, rules and assumptions were identified and justified, and a program confirmed the rules on a roll-over example. I learned that the data type of each item should follow from its meaning (identifier, register, money, total).

TASK 2 — Choose the Data Types

Task / Problem Statement

Select a C data type for every data item found in Task 1, check its size and range on this system with sizeof, limits.h and float.h, show why money must not be stored as float rupees, explain what happens to a 16-bit meter register after 65535, and complete the given program (TODO 1) without using if-else.

Aim

To choose data types that are large enough for a district and that cannot produce rounding or overflow errors, and to prove the choice with measurements.

Concept / Theory

A C type has a size (sizeof) and a range. The standard only guarantees minimum sizes: int at least 16 bits, long at least 32 bits and long long at least 64 bits, so the real limits must be read from limits.h and float.h. Unsigned types use modular arithmetic: when a value passes the maximum it wraps to 0, which is exactly how a 16-bit meter register behaves. float and double store numbers in binary, so a decimal value such as 50.10 cannot be stored exactly and small errors accumulate when many values are added. Integers store paise exactly.

Data item	Type chosen	Size	Range	Sufficient?
Meter reading	unsigned short (meter_t)	2 B	0 to 65535	Yes; matches the 16-bit register
Household number	unsigned int	4 B	0 to 4,294,967,295	Yes; 800,000 houses
Category	char	1 B	−128 to 127	Yes; one letter
Units	unsigned int	4 B	0 to 4,294,967,295	Yes; at most 65,535
Bill / charges (paise)	long	8 B here	±9.2 × 10^18 (min. guaranteed ±2.1 × 10^9)	Yes for one bill
District total (paise)	long long	8 B	±9.2 × 10^18	Yes; about 6.5 × 10^10
Money as float rupees	float	4 B	6 decimal digits	No; rounding error

A float keeps only about 6 decimal digits, so Rs 50.10 repeated 4,000 times drifts away from the true total. Storing paise in an integer removes the problem because integers up to 2^63 are exact. After 65535 the unsigned short register wraps to 0, so a plain subtraction current − previous would be negative after a roll-over; modulo-65536 arithmetic gives the correct consumption.

Procedure / Algorithm

1. Print sizeof and the limits of every candidate type using limits.h and float.h.
2. Add Rs 50.10 4,000 times in a float and in integer paise and compare with the exact total.
3. Show that unsigned short 65535 + 1 wraps to 0.
4. Complete TODO 1 with (meter_t)(current - previous), which needs no if-else.
5. Run the completed program and compare with the expected output.
6. Repeat the money test using double rupees to analyse the alternative.

Program / Source Code

consumption.c (Complete the Code: TODO 1 filled)

```
#include <stdio.h>
#include <limits.h>

/* The meter register is a 16-bit counter: after 65535 it rolls over to 0. */
typedef unsigned short meter_t;

unsigned int consumption(meter_t previous, meter_t current)
{
    // TODO 1: Return the units consumed, correct even when the register has rolled over
    // (current < previous); use unsigned 16-bit arithmetic instead of an if-else
```

```
    return (meter_t)(current - previous);
}

int main(void)
{
    printf("sizeof(meter_t) = %zu bytes, maximum reading = %u\n",
           sizeof(meter_t), (unsigned)USHRT_MAX);
    printf("Consumption: %u units\n", consumption(1200, 1530));
    printf("Consumption: %u units\n", consumption(65400, 120));   /* rolled over */
    return 0;
}
```

datatypes.c

```
/* Task 2 - data-type sizes, ranges, and float-versus-paise evidence */
#include <stdio.h>
#include <limits.h>
#include <float.h>

int main(void)
{
    printf("%-14s %6s  %-26s %s\n", "Type", "Bytes", "Minimum", "Maximum");
    printf("%-14s %6zu  %-26d %d\n",   "char",          sizeof(char), CHAR_MIN, CHAR_MAX);
    printf("%-14s %6zu  %-26d %d\n",   "short",         sizeof(short), SHRT_MIN, SHRT_MAX);
    printf("%-14s %6zu  %-26d %u\n",   "unsigned short",sizeof(unsigned short), 0, (unsigned)USHRT_MAX);
    printf("%-14s %6zu  %-26d %d\n",   "int",           sizeof(int), INT_MIN, INT_MAX);
    printf("%-14s %6zu  %-26ld %ld\n", "long",          sizeof(long), LONG_MIN, LONG_MAX);
    printf("%-14s %6zu  %-26lld %lld\n","long long",     sizeof(long long), LLONG_MIN, LLONG_MAX);
    printf("%-14s %6zu  %-26.6g %.6g\n","float",         sizeof(float), FLT_MIN, FLT_MAX);
    printf("%-14s %6zu  %-26.6g %.6g\n","double",        sizeof(double), DBL_MIN, DBL_MAX);
    printf("float keeps %d decimal digits, double keeps %d\n\n", FLT_DIG, DBL_DIG);

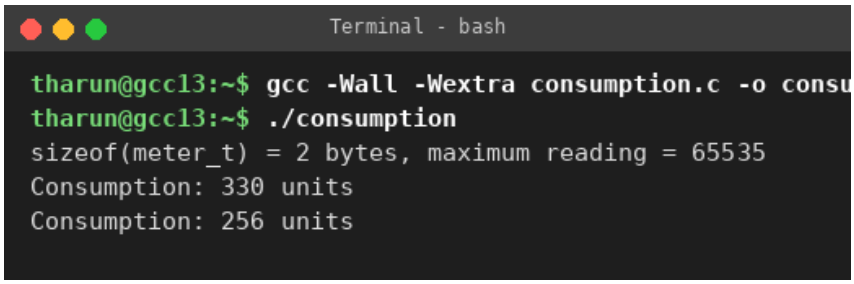
    /* 16-bit meter register after 65535 */
    unsigned short reg = 65535;
    reg = reg + 1;
    printf("unsigned short 65535 + 1 = %u  (wraps to 0)\n\n", (unsigned)reg);

    /* 4,000 bills of Rs 50.10: float rupees versus integer paise */
    float f_total = 0.0f; long p_total = 0;
    for (int i = 0; i < 4000; i++) { f_total += 50.10f; p_total += 5010L; }
    printf("Expected total (4000 x Rs 50.10)   : Rs 200400.00\n");
    printf("float rupees total                 : Rs %.2f\n", f_total);
    printf("long paise total                   : Rs %ld.%02ld\n", p_total/100, p_total%100);
    printf("float error                        : Rs %.2f\n", f_total - 200400.0f);
    return 0;
}
```

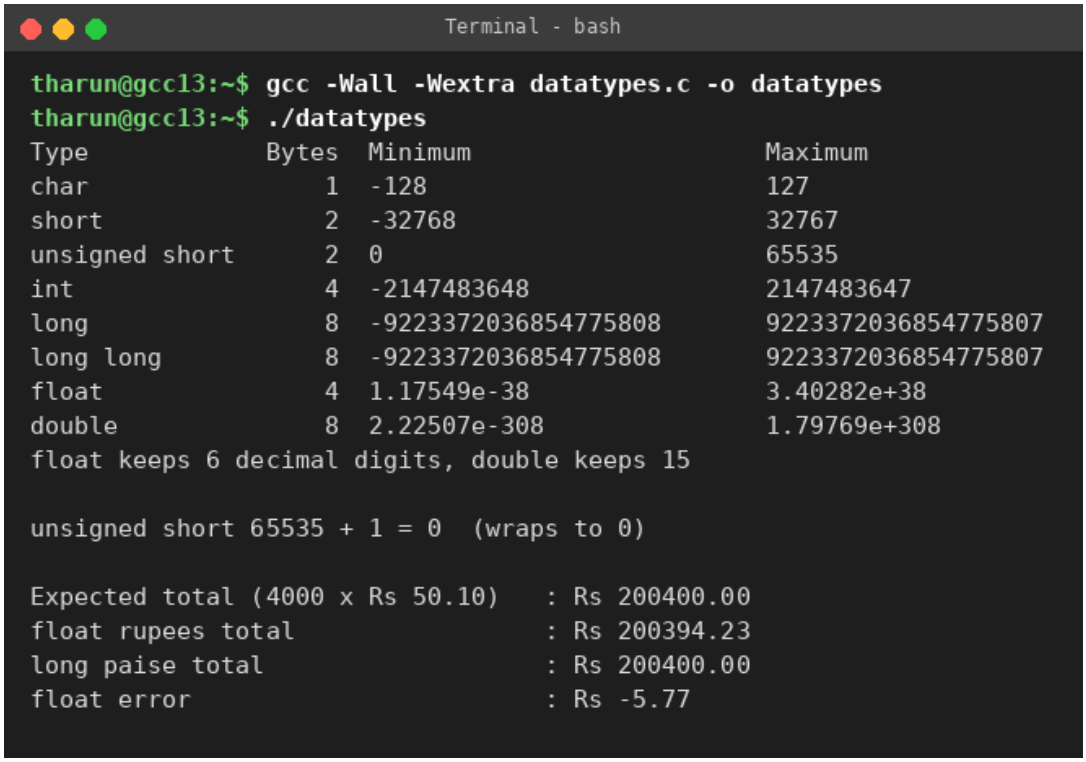
double_rupees.c (alternative concept)

```
/* Alternative concept: money as double rupees, rounded once at the end */
#include <stdio.h>
#include <math.h>
int main(void)
{
    long ns[] = {4000, 60000, 800000};
    for (int k = 0; k < 3; k++) {
        double d = 0.0; long p = 0;
        for (long i = 0; i < ns[k]; i++) { d += 50.10; p += 5010L; }
        long dp = lround(d * 100.0);
        printf("n=%7ld  double -> paise %ld  exact paise %ld  diff %ld  raw error %.3e\n",
               ns[k], dp, p, dp - p, d - p / 100.0);
    }
    return 0;
}
```

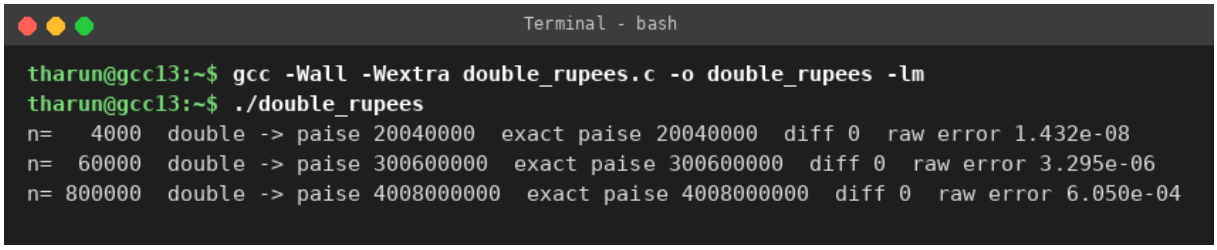
Output Screenshot



Output of consumption.c (matches the expected output)



Output of datatypes.c



Output of double_rupees.c

Analysis / Discussion

consumption(1200, 1530) gives 330 and consumption(65400, 120) gives 256, as expected. The second result is 256 and not negative because 120 − 65400 is calculated as the int −65280, and converting it to the unsigned 16-bit type reduces it modulo 65536: −65280 + 65536 = 256, which is the true number of units (65535 − 65400 + 120 + 1).

The float test gave Rs 200394.23 instead of Rs 200400.00, an error of Rs 5.77, while the paise total was exactly Rs 200400.00. On this system short is 2 bytes, int 4, long 8 and long long 8 bytes, so a district total fits easily in long long.

Alternative Concept and Complexity Analysis

If money were stored as double rupees, each charge would be computed as a double (for example units × 2.25) and rounding to paise would be applied once at the end, using lround(total × 100). The time per addition is O(1) in both designs and the space is 8 bytes per value in both (on a system where long is 4 bytes, paise in long would be smaller, but long long is also 8 bytes). The difference is in the error: for 4,000, 60,000 and 800,000 bills the raw error of the double total grew from 1.4 × 10^-8 to 3.3 × 10^-6 to 6.1 × 10^-4 rupees, so it grows with n. After rounding at the end the paise matched the exact value in all three runs, but the margin shrinks as n grows and comparing two double totals for equality is unsafe. Integer paise has zero error at every n.

Result

Suitable types were selected and verified; unsigned short with modular arithmetic handles the roll-over, and integer paise avoids float rounding (Rs 5.77 error shown). The TODO program printed 330 and 256 units as required.

TASK 3 — Define the Constants

Task / Problem Statement

Define the tariff values and limits as constants so that a tariff revision is made in one place only. Complete TODO 1 of compute_bill() using only the macros defined at the top, run it, then change RATE2_PAISE to 230 and check that every bill changes consistently.

Aim

To keep every tariff value in a single well-named set of constants and to calculate the slab bill in integer paise.

Concept / Theory

A #define constant is replaced by the preprocessor before compilation; it has no type, no storage and no scope. A const variable is a real object: it is type-checked, follows scope rules, can be seen in a debugger, and may occupy memory. An enum defines named integer constants that are type-checked as int and are visible to the debugger but cannot hold a long or a floating value. Names of constants are written in capital letters with a unit suffix, for example RATE2_PAISE, so that the meaning and the unit are clear.

The bill is computed slab by slab: units above SLAB2_UNITS are charged at RATE3_PAISE, units between SLAB1_UNITS + 1 and SLAB2_UNITS at RATE2_PAISE, and the remaining units at RATE1_PAISE. Because only the macros appear in the code, changing RATE2_PAISE changes every bill consistently.

Procedure / Algorithm

1. Define the slab limits, the three rates and the fixed charge as macros at the top of the program.
2. Start with paise = FIXED_CHARGE_PAISE.
3. Charge the units above SLAB2_UNITS at RATE3_PAISE and reduce the units to SLAB2_UNITS.
4. Charge the units above SLAB1_UNITS at RATE2_PAISE and reduce the units to SLAB1_UNITS.
5. Charge the remaining units at RATE1_PAISE and return the bill in paise.
6. Print rupees as paise / 100 and paise as paise % 100; then change RATE2_PAISE to 230 and run again.

Program / Source Code

tariff.c (Complete the Code: TODO 1 filled)

```
#include <stdio.h>

#define SLAB1_UNITS 100        /* first 100 units */
#define SLAB2_UNITS 200        /* units 101 to 200 */
#define RATE1_PAISE 0          /* first slab is free */
#define RATE2_PAISE 225        /* Rs 2.25 per unit */
#define RATE3_PAISE 450        /* Rs 4.50 per unit above 200 */
#define FIXED_CHARGE_PAISE 5000L   /* Rs 50 fixed charge */

/* Returns the bill in paise, so that no rounding error can occur. */
long compute_bill(int units)
{
    long paise = FIXED_CHARGE_PAISE;

    // TODO 1: Add the energy charge slab by slab using the macros: units above
    // SLAB2_UNITS at RATE3_PAISE, units from SLAB1_UNITS+1 to SLAB2_UNITS at
    // RATE2_PAISE, and the rest at RATE1_PAISE
    if (units > SLAB2_UNITS) {
        paise += (long)(units - SLAB2_UNITS) * RATE3_PAISE;
        units = SLAB2_UNITS;
    }
    if (units > SLAB1_UNITS) {
        paise += (long)(units - SLAB1_UNITS) * RATE2_PAISE;
        units = SLAB1_UNITS;
    }
    paise += (long)units * RATE1_PAISE;

    return paise;
}
```

```c
int main(void)
{
    int tests[] = {0, 100, 150, 200, 350};
    int n = sizeof tests / sizeof tests[0];
    for (int i = 0; i < n; i++) {
        long p = compute_bill(tests[i]);
        printf("%3d units -> Rs %ld.%02ld\n", tests[i], p / 100, p % 100);
    }
    return 0;
}
```

const_enum.c (alternative concept)

```c
/* Alternative concept: const variables and enum instead of #define */
#include <stdio.h>
enum { SLAB1_UNITS = 100, SLAB2_UNITS = 200 };          /* integer limits: enum  */
static const int  RATE2_PAISE = 225, RATE3_PAISE = 450; /* typed, has an address */
static const long FIXED_CHARGE_PAISE = 5000L;

static long compute_bill(int units)
{
    long p = FIXED_CHARGE_PAISE;
    if (units > SLAB2_UNITS) { p += (long)(units - SLAB2_UNITS) * RATE3_PAISE; units = SLAB2_UNITS; }
    if (units > SLAB1_UNITS)   p += (long)(units - SLAB1_UNITS) * RATE2_PAISE;
    return p;
}
int main(void)
{
    int t[] = {0, 100, 150, 200, 350};
    for (int i = 0; i < 5; i++) { long p = compute_bill(t[i]); printf("%3d units -> Rs %ld.%02ld\n",
        t[i], p/100, p%100); }
    printf("&RATE2_PAISE = %p  (a const variable has storage; a #define has none)\n", (void
        *)&RATE2_PAISE);
    return 0;
}
```

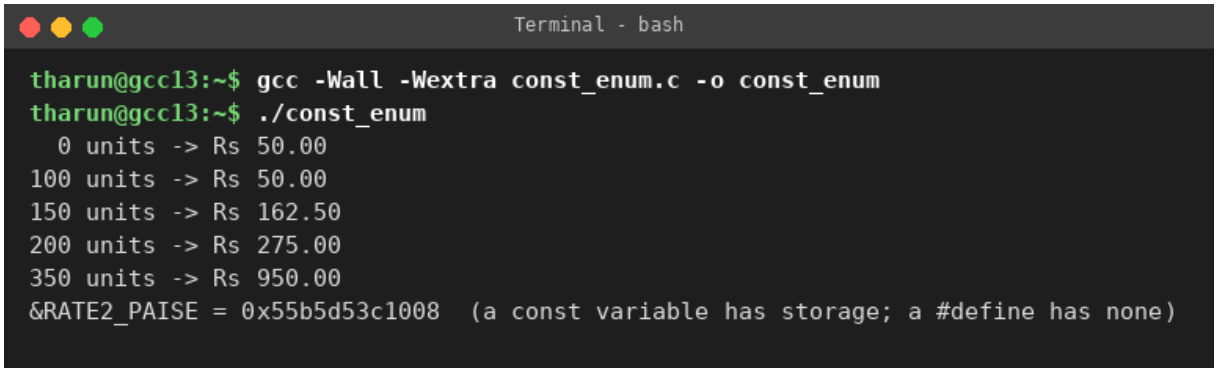
Output Screenshot



Output of tariff.c with RATE2_PAISE = 225 (expected output obtained)



Output after changing RATE2_PAISE to 230



Output of const_enum.c

Analysis / Discussion

With 225 the output is exactly the expected one: 0 units Rs 50.00, 100 units Rs 50.00, 150 units Rs 162.50, 200 units Rs 275.00, 350 units Rs 950.00. After changing only RATE2_PAISE to 230, the bills of 0 and 100 units stay the same (they never reach slab 2), 150 units rises by 50 × 5 = 250 paise to Rs 165.00, and 200 and 350 units both rise by 100 × 5 = 500 paise to Rs 280.00 and Rs 955.00. This shows that all bills changed consistently from one edit.

Alternative Concept and Complexity Analysis

If the constants were const variables or an enum, the program would behave the same. Type checking improves: a const long has a real type and the compiler checks its use, an enum is checked as int, while a #define has no type at all. A debugger shows const variables and enum names, but a #define name disappears after preprocessing. Memory: a #define uses no storage; a const variable may occupy a few bytes (the program printed an address for RATE2_PAISE, so it has storage); an enum constant uses no storage. The running time is not affected in practice, because the compiler replaces a static const integer by its value when optimising; at -O0 a const read is one memory load, which is O(1). My choice is enum for integer limits, const for typed rates in a larger program, and #define only for values that must be used in the preprocessor or in this assignment's fixed code.

Result

TODO 1 was completed using only the macros and produced the expected output. Changing RATE2_PAISE once changed all affected bills correctly, which solves the partial-revision problem.

TASK 4 — Use Macros Safely

Task / Problem Statement

Write macros for small repeated calculations (rupees to paise, larger of two values, square), show how the preprocessor expands them with gcc -E, demonstrate why SQUARE(x) written as x*x fails for SQUARE(a+1), why MAX(a, b) can evaluate an argument twice, and decide when a function is a better choice.

Aim

To write fully parenthesised macros and to understand the pitfalls of macro expansion.

Concept / Theory

A macro is replaced by text before compilation, with no type checking and no evaluation of its arguments. The argument text is pasted in as written, so operator precedence can change the meaning: x*x with x = a+1 becomes a+1*a+1. Every use of a macro parameter should therefore be enclosed in parentheses and the whole expansion should also be enclosed. A macro pastes its argument at each place it appears, so an argument with a side effect, such as i++, can be executed twice. A function evaluates each argument once, is type-checked and can be stepped through in a debugger, so a function (often static inline) is preferable whenever the arguments may have side effects or the calculation is longer than one expression.

Procedure / Algorithm

1. Write RUPEES_TO_PAISE, MAX and SQUARE with parentheses around each parameter and around the whole expression.
2. Write an unsafe BAD_SQUARE(x) as x * x for comparison.
3. Compile and run to compare BAD_SQUARE(a+1) with SQUARE(a+1).
4. Run MAX(i++, j) and compare with a function max_fn(i++, j).
5. Use gcc -E to display the expanded text.
6. Write static inline versions and time the macro and the function.

Program / Source Code

macros.c

```
#include <stdio.h>
#include <time.h>

#define RUPEES_TO_PAISE(r)  ((r) * 100L)
#define MAX(a, b)           ((a) > (b) ? (a) : (b))
#define SQUARE(x)           ((x) * (x))
#define BAD_SQUARE(x)       x * x           /* deliberately unsafe */

static inline int  max_fn(int a, int b)  { return a > b ? a : b; }
static inline long sq_fn(long x)         { return x * x; }

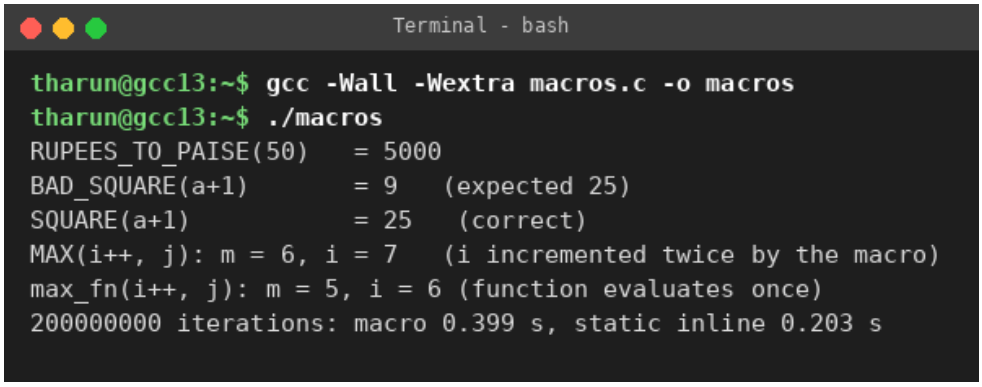
int main(void)
{
    int a = 4;
    printf("RUPEES_TO_PAISE(50)   = %ld\n", RUPEES_TO_PAISE(50));
    printf("BAD_SQUARE(a+1)       = %d   (expected 25)\n", BAD_SQUARE(a + 1));
    printf("SQUARE(a+1)           = %d   (correct)\n", SQUARE(a + 1));

    int i = 5, j = 3;
    int m = MAX(i++, j);
    printf("MAX(i++, j): m = %d, i = %d   (i incremented twice by the macro)\n", m, i);
    i = 5; j = 3;
    m = max_fn(i++, j);
    printf("max_fn(i++, j): m = %d, i = %d (function evaluates once)\n", m, i);

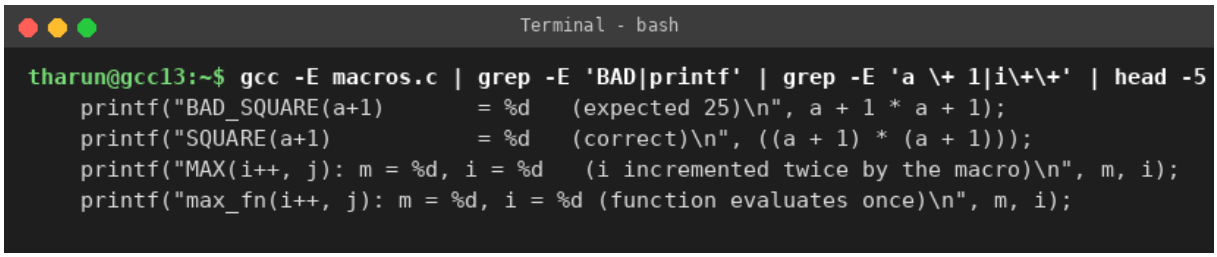
    /* time: macro vs static inline */
    enum { N = 200000000 };
    volatile long sink = 0; clock_t t0 = clock();
    for (long k = 0; k < N; k++) sink += SQUARE(k & 1023);
    double tm = (double)(clock() - t0) / CLOCKS_PER_SEC;
```

```
    t0 = clock();
    for (long k = 0; k < N; k++) sink += sq_fn(k & 1023);
    double tf = (double)(clock() - t0) / CLOCKS_PER_SEC;
    printf("%d iterations: macro %.3f s, static inline %.3f s\n", (int)N, tm, tf);
    return 0;
}
```

Output Screenshot



Output of macros.c



gcc -E evidence (expanded macros)

Analysis / Discussion

BAD_SQUARE(a+1) gave 9 instead of 25 because gcc -E shows it expanded to a + 1 * a + 1, which is 4 + 4 + 1. SQUARE(a+1) expanded to ((a + 1) * (a + 1)) and gave 25. For MAX(i++, j) with i = 5 and j = 3 the expansion is ((i++) > (j) ? (i++) : (j)); i++ appears twice, so the macro returned 6 and i became 7, while the function max_fn returned 5 and left i = 6. RUPEES_TO_PAISE(50) correctly gave 5000.

Alternative Concept and Complexity Analysis

Replacing MAX and SQUARE by static inline functions removes the double evaluation and adds type checking. The time per use is O(1) in both. In my measurement of 200,000,000 iterations at -O0 the macro took 0.399 s and the inline function 0.203 s, because the macro computed its argument expression (k & 1023) twice; with optimisation (-O2) both took 0.075 s, so there is no speed advantage for macros. Space: a macro is copied at every one of its k uses, so the code size grows with k, while a function body exists once (plus k small call instructions, or k copies only if the compiler chooses to inline it). For a calculation used many times, the function is better; macros are suitable for constants and for very small side-effect-free expressions.

Result

Fully parenthesised macros were written and the preprocessor output proved each pitfall. I learned to prefer static inline functions when arguments may have side effects.

TASK 5 — Organise the Program with Header Files and Conditional Compilation

Task / Problem Statement

Move the constants and the function prototype into a header file with include guards, split the program into billing.h, billing.c and main.c, and add a debugging mode that prints intermediate values only when the program is compiled with gcc -DDEBUG.

Aim

To organise a C program into header and source files and to use conditional compilation for debugging.

Concept / Theory

A C program usually has a header file for constants and prototypes, one or more .c files for the functions, and main() as the entry point. Include guards (#ifndef BILLING_H, #define BILLING_H, #endif) make sure that the contents of a header are processed once even if it is included twice, which prevents redefinition errors. #include <file> searches the system include directories and is used for standard headers, while #include "file" searches the current directory first and is used for the program's own headers. With #ifdef DEBUG, the debugging lines are compiled only when DEBUG is defined on the command line (gcc -DDEBUG); otherwise the preprocessor removes them, so the final program contains no debug code.

Procedure / Algorithm

1. Create billing.h with the include guard, the tariff macros, the DBG() macro and the prototype of compute_bill().
2. Define DBG() as printf only when DEBUG is defined, and as an empty statement otherwise.
3. Write billing.c with compute_bill() and DBG() calls for the intermediate values.
4. Write main.c that includes billing.h and prints three bills.
5. Compile normally and with -DDEBUG and compare the outputs.
6. Compare the object size of both builds and time a run-time debug flag.

Program / Source Code

billing.h

```
#ifndef BILLING_H
#define BILLING_H

#include <stdio.h>

#define SLAB1_UNITS 100
#define SLAB2_UNITS 200
#define RATE1_PAISE 0
#define RATE2_PAISE 225
#define RATE3_PAISE 450
#define FIXED_CHARGE_PAISE 5000L

#ifdef DEBUG
#define DBG(...) printf("[DEBUG] " __VA_ARGS__)
#else
#define DBG(...) ((void)0)
#endif

long compute_bill(int units);

#endif /* BILLING_H */
```

billing.c

```
#include "billing.h"
#include "billing.h"   /* second include is harmless because of the include guard */

long compute_bill(int units)
{
    long paise = FIXED_CHARGE_PAISE;
    DBG("units = %d\n", units);
```

```
    if (units > SLAB2_UNITS) { paise += (long)(units - SLAB2_UNITS) * RATE3_PAISE; units = SLAB2_UNITS;
        }
    DBG("after slab 3: paise = %ld\n", paise);
    if (units > SLAB1_UNITS) { paise += (long)(units - SLAB1_UNITS) * RATE2_PAISE; units = SLAB1_UNITS;
        }
    DBG("after slab 2: paise = %ld\n", paise);
    paise += (long)units * RATE1_PAISE;
    return paise;
}
```

main.c

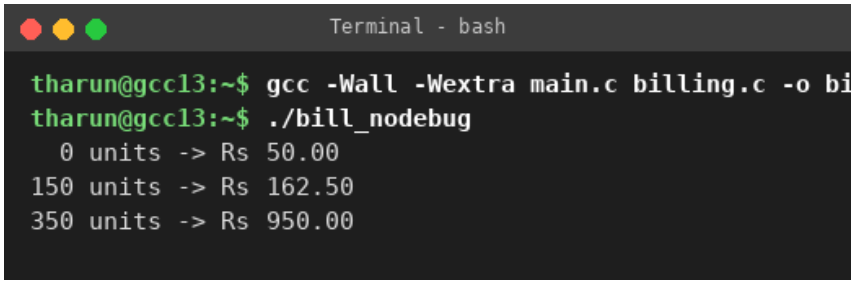
```
#include "billing.h"

int main(void)
{
    int tests[] = {0, 150, 350};
    for (int i = 0; i < 3; i++) {
        long p = compute_bill(tests[i]);
        printf("%3d units -> Rs %ld.%02ld\n", tests[i], p / 100, p % 100);
    }
    return 0;
}
```

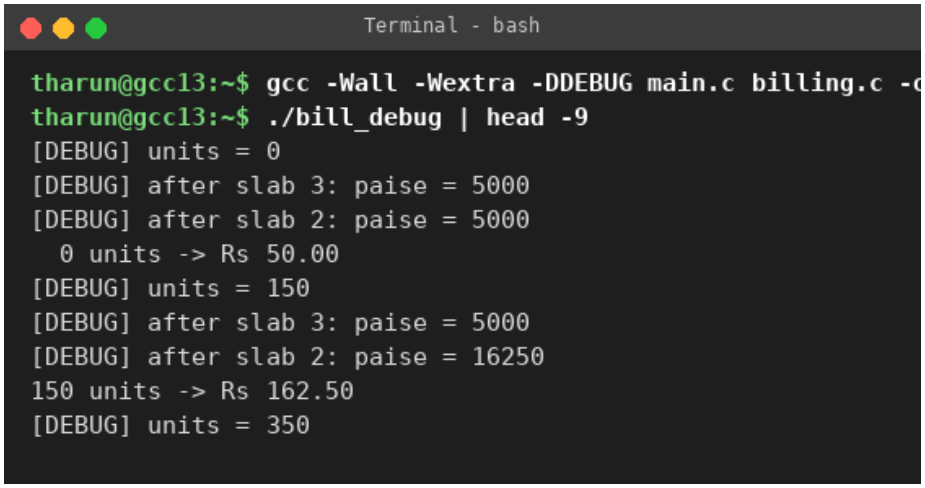
runtime_debug.c (alternative concept)

```
/* Alternative concept: run-time debug flag instead of #ifdef DEBUG */
#include <stdio.h>
#include <time.h>
static int debug = 0;
static long f(long x) { if (debug) printf("x=%ld\n", x); return x + 1; }
#ifdef DEBUG
static long g(long x) { printf("x=%ld\n", x); return x + 1; }
#else
static long g(long x) { return x + 1; }
#endif
int main(void)
{
    enum { N = 300000000 };
    volatile long s = 0; clock_t t0 = clock();
    for (long i = 0; i < N; i++) s += f(i & 7);
    double t1 = (double)(clock() - t0) / CLOCKS_PER_SEC;
    t0 = clock();
    for (long i = 0; i < N; i++) s += g(i & 7);
    double t2 = (double)(clock() - t0) / CLOCKS_PER_SEC;
    printf("run-time flag (if (debug)) : %.3f s for %d calls\n", t1, (int)N);
    printf("compile-time (#ifdef DEBUG): %.3f s for %d calls\n", t2, (int)N);
    return 0;
}
```

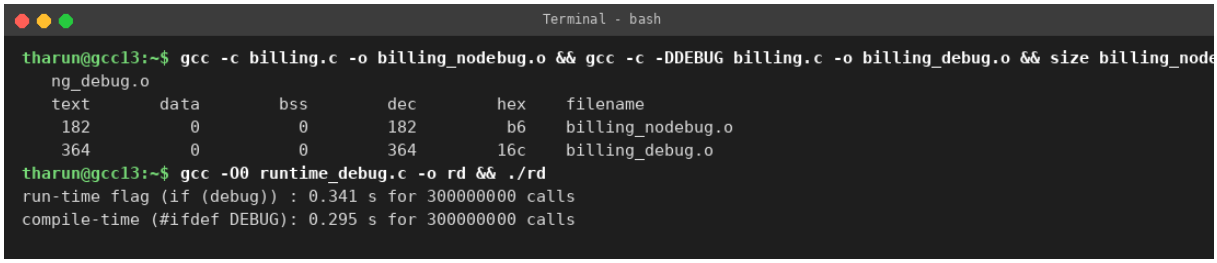
Output Screenshot



Normal build: no debug output



Debug build (gcc -DDEBUG): intermediate values printed



Object size with and without DEBUG, and run-time flag timing

Analysis / Discussion

The normal build prints only the three bills. The -DDEBUG build prints the units and the running total after each slab, for example 16250 paise after slab 2 for 150 units. billing.h is included twice in billing.c on purpose; the include guard prevented any redefinition error.

Alternative Concept and Complexity Analysis

If debugging were controlled by a run-time variable (if (debug) printf(...)), the same executable could switch debugging on and off without recompiling, but the test is executed in every call: an extra O(1) comparison and branch each time. In my measurement of 300,000,000 calls the run-time flag took 0.341 s and the compile-time version 0.295 s, about 15% more. Space: with the run-time flag the debug code and its strings stay in the executable in all cases; with #ifdef DEBUG the text segment of billing.o was 182 bytes without DEBUG and 364 bytes with DEBUG, so the release build contains none of the debug code. Compile-time control is better for the released program; run-time control is more convenient for field diagnosis.

Result

The program was organised into a header and two source files with include guards, and the DEBUG mode worked only when compiled with -DDEBUG.

TASK 6 — Read and Print Data Correctly

Task / Problem Statement

Design the input and output of the program with scanf and printf: use the correct format specifier for each type, show what happens when a wrong specifier is used, detect invalid input such as letters by checking the return value of scanf, and print an amount held in paise as rupees and paise (for example Rs 162.50).

Aim

To read and print values of every type correctly and to reject invalid input safely.

Concept / Theory

printf and scanf use format specifiers that must match the argument type: %d int, %u unsigned int, %ld long, %lld long long, %f float in printf, %lf double in scanf (printf accepts %f for double), %c char and %s string. A wrong specifier is undefined behaviour: for example %d reads only 32 bits of a 64-bit long. scanf returns the number of items it converted successfully; if the user types letters where a number is expected, the return value is smaller than expected and the input must be rejected. An amount in paise is printed as paise / 100 for the rupees and paise % 100 with %02ld for the two digits of paise.

Procedure / Algorithm

1. Read household, previous and current reading with scanf("%u %d %d").
2. Compare the return value of scanf with 3 and reject the input if it differs.
3. Print the bill as Rs %ld.%02ld using paise / 100 and paise % 100.
4. Test a long value printed with %d to see the effect of a wrong specifier.
5. Print one value of each type with its correct specifier.
6. Write the alternative that reads a line with fgets and converts it with strtol.

Program / Source Code

io.c

```
#include <stdio.h>

int main(void)
{
    unsigned int household; int previous, current; long paise = 16250L;
    printf("Enter household, previous, current: ");
    int n = scanf("%u %d %d", &household, &previous, &current);
    if (n != 3) { printf("Invalid input: scanf returned %d (expected 3)\n", n); return 1; }
    printf("Read household=%u previous=%d current=%d (scanf returned %d)\n", household, previous,
        current, n);
    printf("Bill: Rs %ld.%02ld\n", paise / 100, paise % 100);
    return 0;
}
```

wrongspec.c

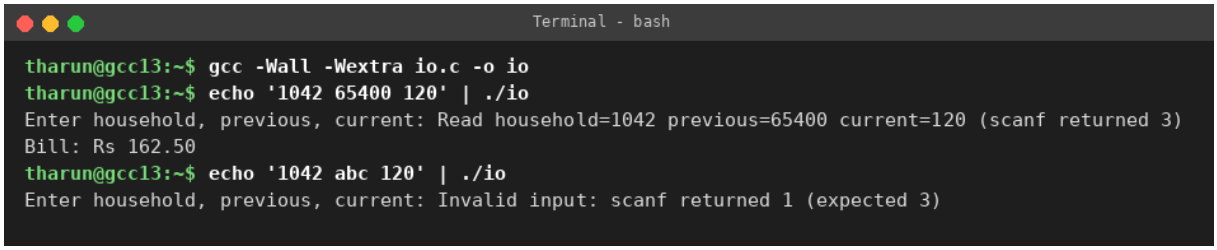
```
#include <stdio.h>
int main(void)
{
    long big = 5000000000L; long long ll = 123456789012LL; float f = 162.5f; double d = 162.5;
    printf("correct %%ld  : %ld\n", big);
    printf("wrong   %%d   : %d   (long printed with %%d)\n", big);
    printf("%%lld %%f %%lf %%c %%s %%u : %lld %f %lf %c %s %u\n", ll, f, d, 'X', "text", 65535u);
    printf("print rupees and paise : Rs %ld.%02ld\n", 16250L / 100, 16250L % 100);
    printf("negative paise -7 -> Rs %ld.%02ld (needs sign handling)\n", -7L / 100, -7L % 100);
    return 0;
}
```

fgets_version.c (alternative concept)

```
/* Alternative concept: fgets() + strtol() instead of scanf() */
#include <stdio.h>
```

```c
#include <stdlib.h>
#include <errno.h>
int main(void)
{
    char line[64], *end;
    printf("Enter a meter reading: ");
    if (!fgets(line, sizeof line, stdin)) return 1;
    errno = 0;
    long v = strtol(line, &end, 10);
    if (end == line || (*end != '\n' && *end != '\0') || errno || v < 0 || v > 65535) {
        printf("Rejected line: %s", line);
        return 1;
    }
    printf("Accepted reading = %ld\n", v);
    return 0;
}
```

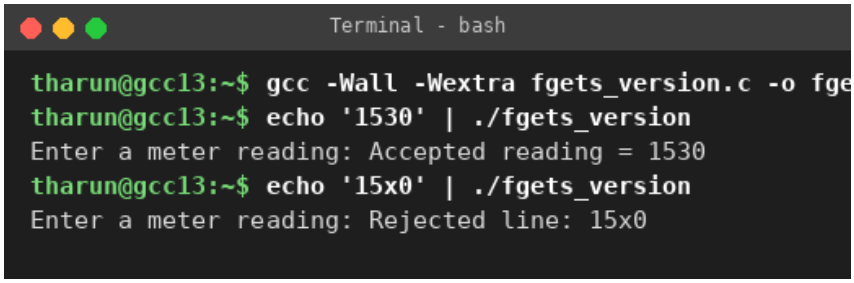
Output Screenshot



Output of io.c with valid and invalid input



Output of wrongspec.c (compiled with -Wno-format)



Output of fgets_version.c

Analysis / Discussion

For the valid line scanf returned 3 and the bill Rs 162.50 was printed from 16250 paise. For "1042 abc 120" scanf returned 1, so the program printed an error message instead of using garbage. The wrong-specifier test printed 5000000000 correctly with %ld but 705032704 with %d; this is the lower 32 bits of the number and shows why the specifier must match. The last test shows an edge case of the paise formula: for a negative amount of −7 paise the output was Rs 0.-7, so a sign must be handled separately if refunds are ever printed.

Alternative Concept and Complexity Analysis

If each line is read with fgets() and converted with strtol() or sscanf(), the whole line is stored in a buffer of fixed size (64 bytes here, so O(1) space) and then scanned once, so the time is O(L) for a line of length L, the same order as scanf, which also reads each character once. The difference is recovery from invalid input: after scanf fails on letters, the bad characters stay in the input buffer and must be removed in a loop, otherwise every later scanf fails again; with fgets the whole line is already consumed, so the next line starts clean. In the test, strtol accepted "1530" and rejected "15x0" because the end pointer did not reach the end of the line. fgets with strtol is therefore the safer design for a billing office.

Result

Input and output were designed with correct specifiers, invalid input was detected from the return value of scanf, and the bill was printed correctly as rupees and paise.

TASK 7 — Control Type Conversion and Precision

Task / Problem Statement

Identify the places where values of different types are combined in the billing program, demonstrate at least three expressions that give an unexpected result and explain them, and state a consistent rounding rule for converting a fractional charge to paise.

Aim

To understand implicit conversion, integer division, casting and rounding so that no hidden conversion changes a bill.

Concept / Theory

In C, operands of different types are converted automatically. Small integer types (char, short) are promoted to int before arithmetic; when the operands still differ, the usual arithmetic conversions bring them to a common type, so a signed int mixed with an unsigned int becomes unsigned. Division of two integers discards the fraction. An explicit cast from a wider to a narrower type, such as long to int, keeps only the low bits. Converting a double to an integer truncates toward zero, so a value that is slightly below the true value (like 1998.9999999) loses a paisa. The rule used in this assignment: keep all money in integer paise; where a fractional paise arises (Task 12), round once, half up, at the end.

Places where types meet in the billing program: units (unsigned int) with a rate (int) and the result stored in long; meter_t (unsigned short) subtraction promoted to int; int units multiplied by a rate that is cast to long to avoid 32-bit overflow; and the final paise to rupees conversion.

Procedure / Algorithm

1. List the expressions in the program where different types are combined.
2. Test units / 100 * 2.25 against units / 100.0 * 2.25.
3. Test (int) of a large long and the promotion of an unsigned short.
4. Test comparison of -1 with 1u and truncation of 19.99 * 100.
5. Explain each result and decide the rounding rule.
6. Measure float accumulation against integer paise for 4,000, 60,000 and 800,000 bills.

Program / Source Code

conversions.c

```
#include <stdio.h>
#include <math.h>

int main(void)
{
    int units = 150;
    printf("1. units / 100 * 2.25      = %.2f   (integer division first: 150/100 = 1)\n", units / 100 *
        2.25);
    printf("   units / 100.0 * 2.25    = %.2f\n", units / 100.0 * 2.25);

    long big = 5000000000L;
    printf("2. (int)5000000000L        = %d   (narrowing cast loses the high bits)\n", (int)big);

    unsigned short a = 65535;
    printf("3. a + 1 (a = 65535 ushort)= %d   (promoted to int, so no wrap)\n", a + 1);
    printf("   (unsigned short)(a + 1) = %u\n", (unsigned)(unsigned short)(a + 1));

    int  s = -1; unsigned int u = 1;
    printf("4. -1 < 1u                 = %d   (signed converted to unsigned: -1 becomes 4294967295)\n",
        s < u);

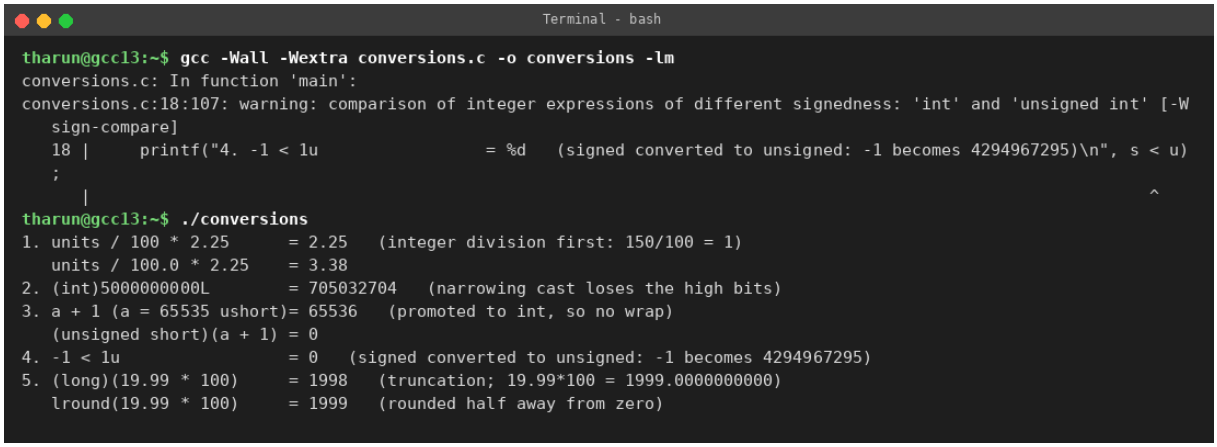
    double r = 19.99;
    printf("5. (long)(19.99 * 100)     = %ld   (truncation; 19.99*100 = %.10f)\n", (long)(r * 100), r *
        100);
    printf("   lround(19.99 * 100)     = %ld   (rounded half away from zero)\n", lround(r * 100));
    return 0;
```

```
}
```

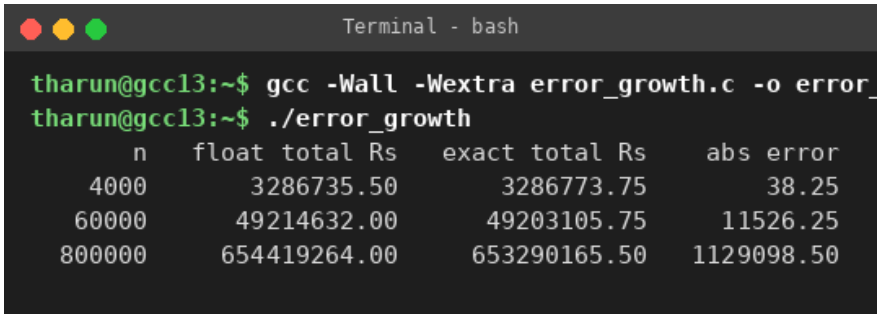
error_growth.c (alternative concept)

```
/* Alternative concept: float arithmetic rounded once at the end vs integer paise */
#include <stdio.h>
#include <math.h>
#include <time.h>
static long bill_paise(int u)
{ long p = 5000; if (u > 200) { p += (long)(u - 200) * 450; u = 200; } if (u > 100) p += (long)(u - 100)
     * 225; return p; }
static float bill_float(int u)
{ float p = 50.0f; if (u > 200) { p += (u - 200) * 4.50f; u = 200; } if (u > 100) p += (u - 100) *
    2.25f; return p; }
int main(void)
{
    long ns[] = {4000, 60000, 800000};
    printf("%8s %16s %16s %12s\n", "n", "float total Rs", "exact total Rs", "abs error");
    for (int k = 0; k < 3; k++) {
        float ft = 0.0f; long pt = 0; unsigned seed = 12345;
        for (long i = 0; i < ns[k]; i++) {
            seed = seed * 1103515245u + 12345u; int u = (seed >> 16) % 600;
            ft += bill_float(u) + 0.07f; pt += bill_paise(u) + 7;
        }
        printf("%8ld %16.2f %13ld.%02ld %12.2f\n", ns[k], ft, pt / 100, pt % 100, fabs(ft - pt /
            100.0));
    }
    return 0;
}
```

Output Screenshot



Output of conversions.c (the sign-compare warning is intentional)



Output of error_growth.c

Analysis / Discussion

- units / 100 * 2.25 gave 2.25, not 3.38: 150 / 100 is integer division and equals 1 before the multiplication; with 100.0 the result is correct.

- (int)5000000000L gave 705032704: the cast keeps only the lowest 32 bits, so a large paise total must never be cast to int.
- a + 1 for an unsigned short 65535 gave 65536, not 0, because a is promoted to int; the wrap to 0 happens only when the result is stored back into the 16-bit type.
- -1 < 1u gave 0 (false): the signed −1 is converted to unsigned 4294967295, and gcc -Wall -Wextra warned about it.
- (long)(19.99 * 100) gave 1998 because 19.99 * 100 is really 1998.99999999999; truncation lost one paisa, while lround gave the correct 1999.

Alternative Concept and Complexity Analysis

If everything were calculated in float and rounded only once at the end, the code would be the same shape with float variables, but the error of the total grows with n. In my measurement the float total differed from the exact total by Rs 38.25 for 4,000 bills, Rs 11,526.25 for 60,000 bills and Rs 1,129,098.50 for 800,000 bills. The error grows faster than the number of bills because a float has only 24 bits of precision; once the total exceeds 16,777,216 it can no longer represent every paisa, and each addition is rounded. Time per operation is O(1) in both designs and a float needs 4 bytes against 8 bytes for long here, but the saving of 4 bytes does not justify a wrong total.

Result

Every place where types are combined was identified; five surprising expressions were demonstrated and explained, and the rule of integer paise with a single half-up rounding was fixed.

TASK 8 — Trace the Compilation Process

Task / Problem Statement

Follow a billing program through the compilation stages (gcc -E, -S, -c and linking), show the output of each stage, give one compile-time, one link-time and one run-time error from my work, explain the warnings given by gcc -Wall -Wextra and how they were fixed, and compare building one file with building several files.

Aim

To understand what happens at each stage of compilation and how to classify errors and warnings.

Concept / Theory

Compilation has four stages. The preprocessor (gcc -E) removes comments, includes header files and replaces macros. The compiler (gcc -S) translates the C text into assembly. The assembler (gcc -c) creates an object file with machine code but unresolved references. The linker joins object files and libraries into the executable. A compile-time error is a violation of C syntax or types found by the compiler; a link-time error is an unresolved symbol found by the linker (undefined reference); a run-time error happens while the program is running (for example division by zero). Warnings do not stop compilation but point to code that is probably wrong, and -Wall -Wextra enable most of them.

Procedure / Algorithm

1. Run gcc -E and compare the line counts of the source and the preprocessed file.
2. Run gcc -S and count the assembly lines; run gcc -c and inspect the object file.
3. Link the object file and run the executable.
4. Create one compile-time, one link-time and one run-time error and record the messages.
5. Compile warn.c with -Wall -Wextra, fix every warning and compile again.
6. Build a three-file version with a shared header and measure a full and an incremental build.

Program / Source Code

sample.c (program traced through the stages)

```
#include <stdio.h>
#define FIXED 5000L
#define RATE 225
long bill(int units) { return FIXED + (long)(units > 100 ? units - 100 : 0) * RATE; }
int main(void) { printf("%ld\n", bill(150)); return 0; }
```

warn.c (program with warnings)

```
#include <stdio.h>
int main(int argc, char **argv)
{
    int unused = 3; long total = 5000L; unsigned u = 7; int s = -1;
    printf("%d\n", total);
    if (s < u) puts("less");
    return 0;
}
```

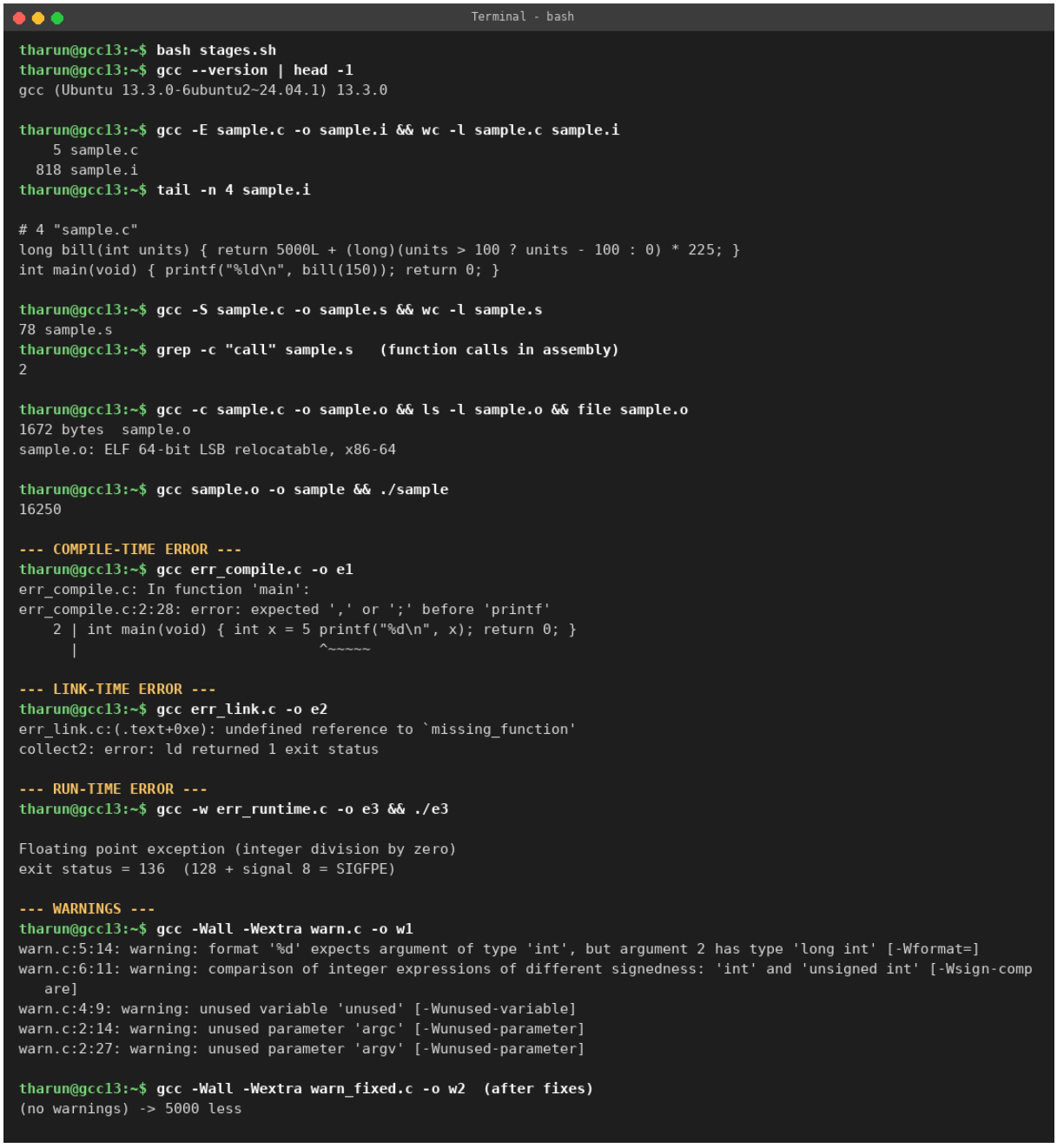
warn_fixed.c (warnings resolved)

```
#include <stdio.h>
int main(void)
{
    long total = 5000L; unsigned u = 7; int s = -1;
    printf("%ld\n", total);
    if (s < 0 || (unsigned)s < u) puts("less");
    return 0;
}
```

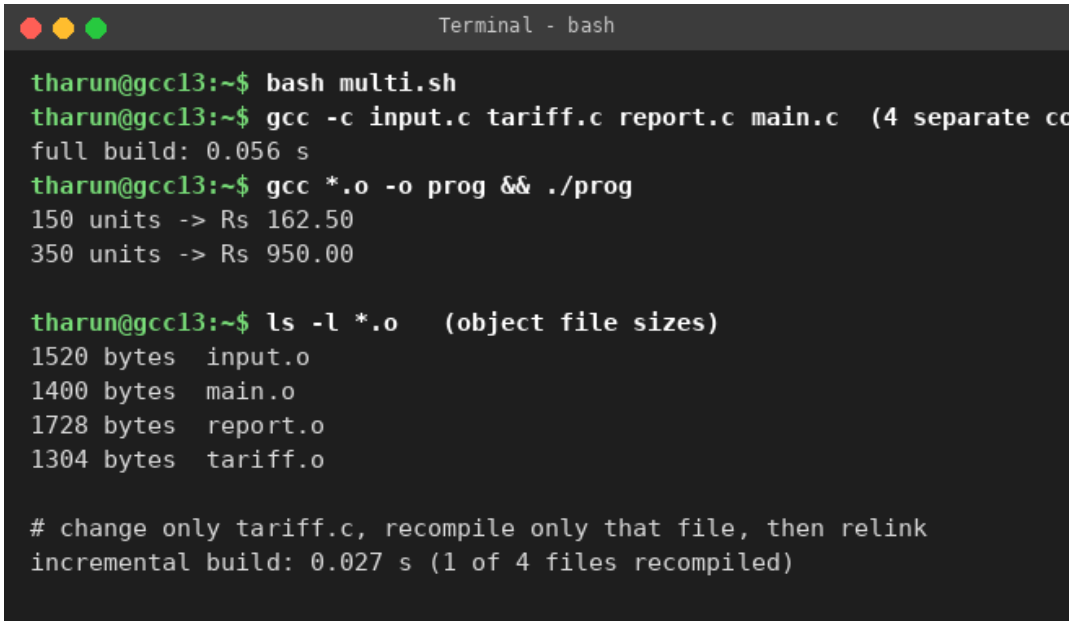
multi/tariff.c (alternative concept; tariff.h, report.c, input.c, main.c are in the repo folder)

```
#include "tariff.h"
long compute_bill(int u){ long p=5000; if(u>200){p+=(long)(u-200)*450;u=200;}
    if(u>100)p+=(long)(u-100)*225; return p; }
```

Output Screenshot



Output of each compilation stage, errors and warnings



Separate compilation of a multi-file build

Analysis / Discussion

The 5-line source grew to 818 lines after gcc -E because the contents of stdio.h were inserted; the comments were removed and the macros FIXED and RATE were replaced by 5000L and 225 in the function. gcc -S produced 78 lines of assembly, gcc -c produced a 1672-byte ELF relocatable object file, and linking produced an executable that printed 16250 paise.

- Compile-time error: "expected ',' or ';' before 'printf'" (missing semicolon after int x = 5).
- Link-time error: "undefined reference to 'missing_function'" followed by "ld returned 1 exit status".
- Run-time error: integer division by zero gave a floating point exception (SIGFPE), exit status 136.
- Warnings: -Wformat (%d used for a long) fixed by %ld; -Wsign-compare (int compared with unsigned) fixed by checking s < 0 first and casting; -Wunused-variable removed; -Wunused-parameter removed by using main(void). After the fixes gcc printed no warnings.

Alternative Concept and Complexity Analysis

If the program is split into input.c, tariff.c, report.c and main.c with a shared tariff.h, each .c file is compiled separately to its own .o file and the linker joins them. A full build compiled four files (0.056 s). After changing only tariff.c, only that file was recompiled and relinked (0.027 s), about half the time; in a one-file program the whole file must be recompiled after every change, so the compile time is O(size of the whole program) against O(size of the changed file) plus the link time. When the shared header changes, every file that includes it must be recompiled. Space: the four object files used 1520, 1400, 1728 and 1304 bytes (5952 bytes) in addition to the executable, while a single-file build needs only one object file.

Result

The program was followed through the preprocessor, compiler, assembler and linker; one error of each kind was produced and classified, and all warnings from -Wall -Wextra were resolved.

TASK 9 — Implement and Test the Billing Program

Task / Problem Statement

Implement the complete billing program and test it with normal bills in each slab, the boundary cases 100 units, 200 units and 0 units, a meter that has rolled over from 65535 to 0, invalid input (letters, a negative reading, a current reading smaller than the previous one without roll-over), and a set of 5 bills whose total is calculated by hand.

Aim

To build a complete, validated billing program that is correct to the paisa for valid input and rejects invalid input.

Concept / Theory

The program reads one line per household (number, category, previous reading, current reading) with fgets and sscanf, so a bad line never corrupts the next one. The readings are first read as int so that a negative value can be detected, then checked against 0 and 65535 and converted to the 16-bit meter type. Consumption is calculated by unsigned 16-bit subtraction. If the current reading is smaller than the previous one, the case is accepted as a roll-over only when the wrapped consumption is at most 5000 units; otherwise it is rejected. The bill is computed in integer paise by the slab method of Task 3 and all totals are accumulated in long long.

Procedure / Algorithm

1. Read a line with fgets and parse it with sscanf; reject the line if sscanf does not return 4.
2. Reject negative readings and readings above 65535.
3. Compute units with (meter_t)(current - previous).
4. If current < previous and units > 5000, reject the line as a reading error without roll-over.
5. Compute the bill in paise by slabs and print units, energy, fixed charge and total in rupees and paise.
6. Accumulate the grand total in long long and print the totals; run the 12 test lines and the 5 hand-calculated bills.

Program / Source Code

billing.c

```
/* Task 9 - Complete billing program: reads "household category previous current" lines */
#include <stdio.h>
#include <string.h>

#define SLAB1_UNITS        100
#define SLAB2_UNITS        200
#define RATE1_PAISE        0
#define RATE2_PAISE        225
#define RATE3_PAISE        450
#define FIXED_CHARGE_PAISE 5000L
#define MAX_READING        65535
#define MAX_PLAUSIBLE_UNITS 5000          /* a roll-over is accepted only if the wrapped result is
    plausible */

typedef unsigned short meter_t;

static unsigned int consumption(meter_t previous, meter_t current)
{
    return (meter_t)(current - previous);
}

static void rs(char *buf, size_t n, long paise) { snprintf(buf, n, "Rs %ld.%02ld", paise / 100, paise %
    100); }

static long compute_bill(int units)
{
    long paise = FIXED_CHARGE_PAISE;
    if (units > SLAB2_UNITS) { paise += (long)(units - SLAB2_UNITS) * RATE3_PAISE; units = SLAB2_UNITS;
        }
    if (units > SLAB1_UNITS) { paise += (long)(units - SLAB1_UNITS) * RATE2_PAISE; units = SLAB1_UNITS;
```

```
        }
    paise += (long)units * RATE1_PAISE;
    return paise;
}

int main(void)
{
    char line[128];
    unsigned int house; char cat; int prev, curr;
    int ok = 0, bad = 0; long long grand = 0;

    printf("%-6s %-9s %-8s %-10s %-10s %s\n", "House", "Units", "Energy", "Fixed", "Total", "Note");
    while (fgets(line, sizeof line, stdin)) {
        if (sscanf(line, "%u %c %d %d", &house, &cat, &prev, &curr) != 4) {
            line[strcspn(line, "\n")] = '\0';
            printf("REJECTED  \"%s\"  -> not numeric / incomplete\n", line); bad++; continue;
        }
        if (prev < 0 || curr < 0) { printf("REJECTED  house %u -> negative reading\n", house); bad++;
            continue; }
        if (prev > MAX_READING || curr > MAX_READING) { printf("REJECTED  house %u -> reading above
            %d\n", house, MAX_READING); bad++; continue; }
        unsigned int units = consumption((meter_t)prev, (meter_t)curr);
        int rolled = curr < prev;
        if (rolled && units > MAX_PLAUSIBLE_UNITS) {
            printf("REJECTED  house %u -> current < previous without roll-over\n", house); bad++;
                continue;
        }
        long total = compute_bill((int)units), energy = total - FIXED_CHARGE_PAISE;
        char e[24], f[24], t[24];
        rs(e, sizeof e, energy); rs(f, sizeof f, FIXED_CHARGE_PAISE); rs(t, sizeof t, total);
        printf("%-6u %-9u %-10s %-10s %-10s %s\n", house, units, e, f, t, rolled ? "roll-over" : "");
        grand += total; ok++;
    }
    printf("Bills issued: %d, rejected: %d, total billed: Rs %lld.%02lld\n", ok, bad, grand / 100, grand
         % 100);
    return 0;
}
```

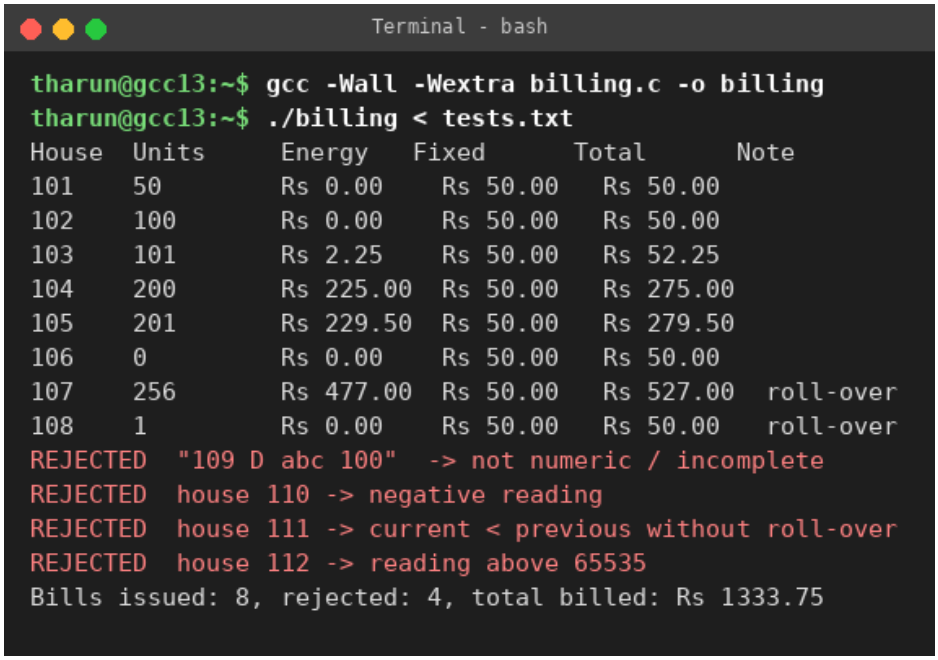
slab_array.c (alternative concept)

```
/* Alternative concept: slab limits and rates in arrays, bill computed by a loop */
#include <stdio.h>
#include <time.h>
#define S 3
static const int  limit[S] = {100, 200, 1 << 30};
static const long rate [S] = {0, 225, 450};

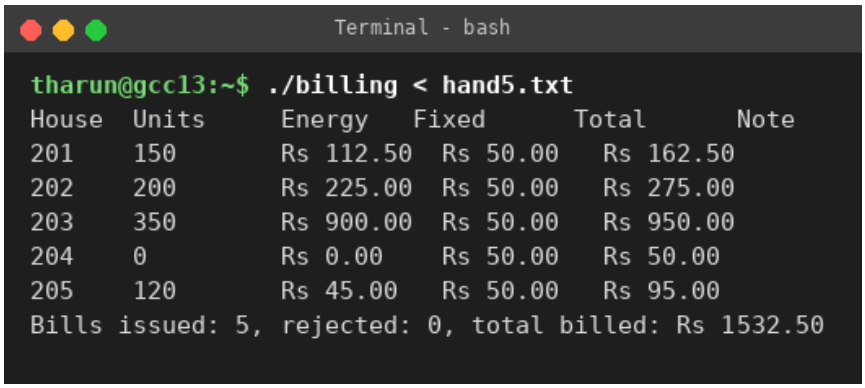
static long bill_loop(int units)
{
    long paise = 5000, prev = 0;
    for (int i = 0; i < S && units > prev; i++) {
        long in_slab = (units < limit[i] ? units : limit[i]) - prev;
        paise += in_slab * rate[i]; prev = limit[i];
    }
    return paise;
}
static long bill_if(int u)
{ long p = 5000; if (u > 200) { p += (long)(u - 200) * 450; u = 200; } if (u > 100) p += (long)(u - 100)
     * 225; return p; }

int main(void)
{
    int t[] = {0, 100, 150, 200, 350}; int bad = 0;
    for (int i = 0; i < 5; i++) { if (bill_loop(t[i]) != bill_if(t[i])) bad++; printf("%3d units -> Rs
        %ld.%02ld\n", t[i], bill_loop(t[i])/100, bill_loop(t[i])%100); }
    printf("loop version agrees with if-version on all 5 tests: %s\n", bad ? "NO" : "YES");
    enum { N = 100000000 }; volatile long s = 0; clock_t t0 = clock();
    for (int i = 0; i < N; i++) s += bill_if(i & 511);
    double a = (double)(clock() - t0) / CLOCKS_PER_SEC; t0 = clock();
    for (int i = 0; i < N; i++) s += bill_loop(i & 511);
    double b = (double)(clock() - t0) / CLOCKS_PER_SEC;
    printf("%d bills: if-version %.3f s, loop-over-slabs version %.3f s\n", (int)N, a, b);
    return 0;
}
```

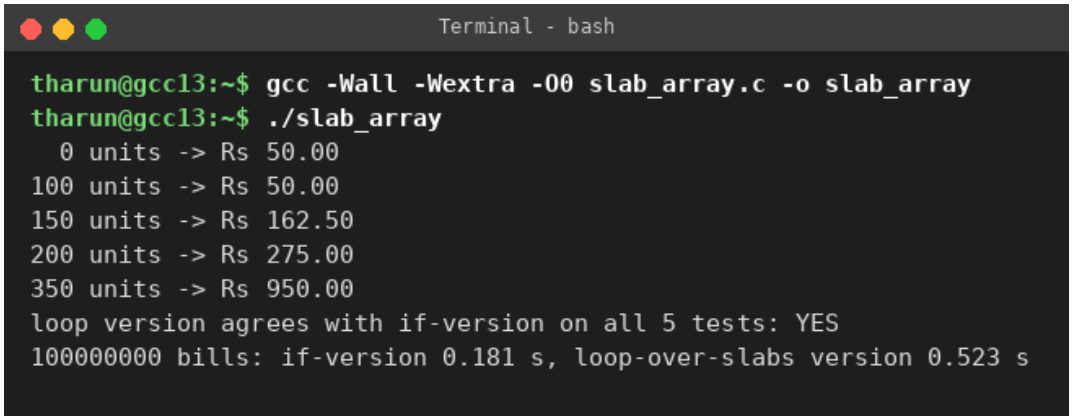
Output Screenshot



Output for the 12 test lines (tests.txt)



Output for the 5 hand-calculated bills (hand5.txt)



Output of slab_array.c

Analysis / Discussion

All eight valid lines gave the expected bills: 50 units Rs 50.00, 100 units Rs 50.00, 101 units Rs 52.25, 200 units Rs 275.00, 201 units Rs 279.50, 0 units Rs 50.00, and the roll-overs 65400 → 120 (256 units, Rs 527.00) and

65535 → 0 (1 unit, Rs 50.00). The four invalid lines (letters, negative reading, current smaller than previous without roll-over, reading above 65535) were all rejected with a reason. For the hand-calculated set of 150, 200, 350, 0 and 120 units, the by-hand bills are Rs 162.50, 275.00, 950.00, 50.00 and 95.00, whose total is Rs 1532.50, and the program printed the same total. The expected values were also confirmed by an independent Python calculation.

Alternative Concept and Complexity Analysis

If the slab limits and rates were stored in two arrays and the bill computed by a loop over s slabs, the code would loop over the slabs and charge min(units, limit[i]) − previous limit at rate[i]. The time per bill is O(s), against a fixed number of tests, that is O(1), for the if-version; with s = 3 the measured times for 100,000,000 bills were 0.523 s (loop) and 0.181 s (if). The extra space is two arrays of s numbers, which is O(s). The array version is easier to change: adding a fourth slab means adding one entry to each array, while the if-version needs new code. For a tariff of only three slabs the if-version is faster; for a tariff that changes often the array version (or a table read from a file) is easier to maintain.

Result

The billing program gave correct bills in every slab and boundary case, handled the roll-over correctly, rejected all invalid input, and matched the hand-calculated total of Rs 1532.50.

TASK 10 — Compare Expectations with Execution

Task / Problem Statement

Compare what was expected from the type and constant design with what actually happened: whether the float and paise versions of the total agree for 4,000, 60,000 and 800,000 bills, whether the int, long and long long totals overflow at district scale, and how sizeof and the ranges on this system compare with the minimum sizes guaranteed by the C standard.

Aim

To test the design against the expectations at village, taluk and district scale and to explain every difference.

Concept / Theory

The C standard guarantees only minimum sizes: short and int at least 16 bits, long at least 32 bits, long long at least 64 bits, float at least 6 and double at least 10 decimal digits. The real sizes depend on the system. Signed integer overflow is undefined behaviour, so the 32-bit test was done with unsigned wrap-around arithmetic and then shown as an int. A float has a 24-bit significand: above 2^24 = 16,777,216 it can no longer represent every multiple of 0.25, and above 2^25 the spacing between neighbouring floats is 4. My expectation was that the float and paise totals would agree for the small village and drift apart for larger n, that a 32-bit int would overflow at district scale, and that long long would be safe.

Procedure / Algorithm

1. Generate 4,000, 60,000 and 800,000 pseudo-random monthly units (0 to 599) with a fixed seed.
2. Accumulate the bills in a float (rupees) and in a long long (paise) and compare.
3. Accumulate the 800,000 bills in a 32-bit int, a long and a long long.
4. Print sizeof and the limits of the types and compare with the standard minimums.
5. Accumulate the same bills in a double and compare with the exact total.
6. Explain every difference between expected and observed behaviour.

Program / Source Code

compare.c

```
/* Task 10 - expected versus observed: float, paise, and int/long/long long totals */
#include <stdio.h>
#include <limits.h>
#include <float.h>

static long bill_paise(int u)
{ long p = 5000; if (u > 200) { p += (long)(u - 200) * 450; u = 200; } if (u > 100) p += (long)(u - 100)
     * 225; return p; }

int main(void)
{
    long ns[] = {4000, 60000, 800000};
    printf("%8s | %18s | %16s | %10s\n", "bills", "paise total (Rs)", "float total (Rs)", "difference");
    for (int k = 0; k < 3; k++) {
        float ft = 0.0f; long long pt = 0; unsigned seed = 2024;
        for (long i = 0; i < ns[k]; i++) {
            seed = seed * 1103515245u + 12345u; int u = (seed >> 16) % 600;
            pt += bill_paise(u); ft += bill_paise(u) / 100.0f;
        }
        printf("%8ld | %15lld.%02lld | %16.2f | %10.2f\n", ns[k], pt / 100, pt % 100, ft, ft - pt /
            100.0);
    }

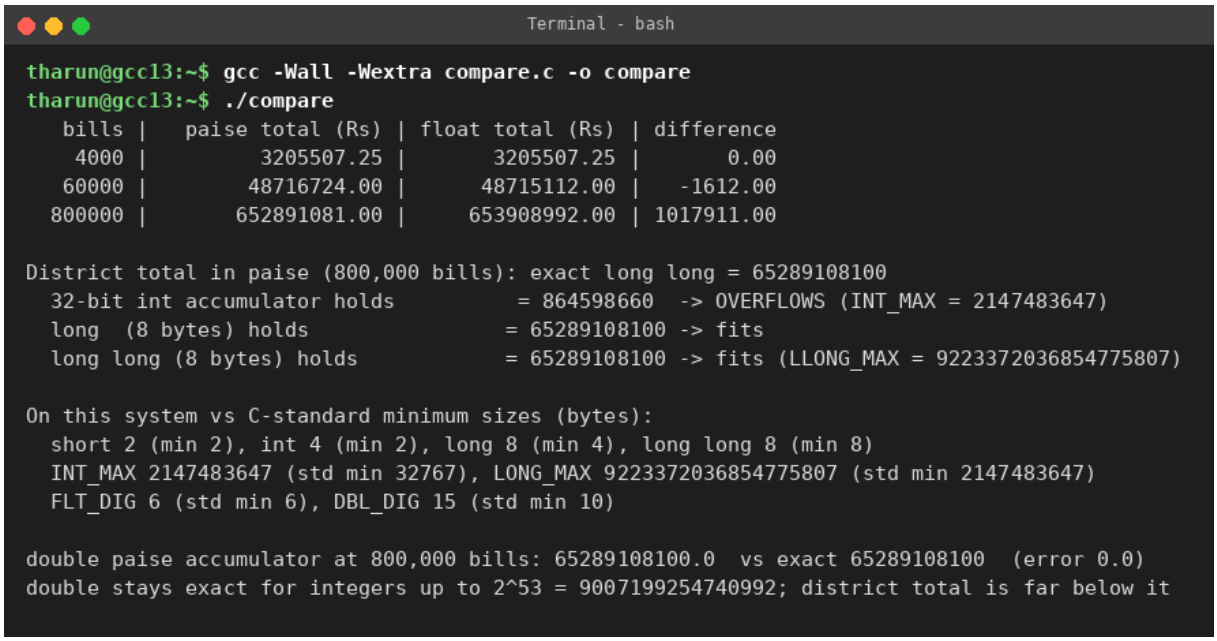
    /* integer accumulators at district scale (800,000 bills), computed with wrap-around arithmetic */
    unsigned int  w32 = 0; long long t64 = 0; unsigned seed = 2024;
    for (long i = 0; i < 800000; i++) {
        seed = seed * 1103515245u + 12345u; int u = (seed >> 16) % 600;
        long b = bill_paise(u); w32 += (unsigned int)b; t64 += b;
    }
    printf("\nDistrict total in paise (800,000 bills): exact long long = %lld\n", t64);
```

```
    printf("  32-bit int accumulator holds          = %d  -> %s\n", (int)w32, t64 > INT_MAX ? "OVERFLOWS
         (INT_MAX = 2147483647)" : "fits");
    printf("  long  (%zu bytes) holds                = %ld -> %s\n", sizeof(long), (long)t64, t64 >
        LONG_MAX ? "OVERFLOWS" : "fits");
    printf("  long long (%zu bytes) holds            = %lld -> fits (LLONG_MAX = %lld)\n", sizeof(long
        long), t64, LLONG_MAX);

    printf("\nOn this system vs C-standard minimum sizes (bytes):\n");
    printf("  short %zu (min 2), int %zu (min 2), long %zu (min 4), long long %zu (min 8)\n",
           sizeof(short), sizeof(int), sizeof(long), sizeof(long long));
    printf("  INT_MAX %d (std min 32767), LONG_MAX %ld (std min 2147483647)\n", INT_MAX, LONG_MAX);
    printf("  FLT_DIG %d (std min 6), DBL_DIG %d (std min 10)\n", FLT_DIG, DBL_DIG);

    /* double accumulator */
    double dt = 0.0; seed = 2024; long long exact = 0;
    for (long i = 0; i < 800000; i++) { seed = seed * 1103515245u + 12345u; int u = (seed >> 16) % 600;
        dt += (double)bill_paise(u); exact += bill_paise(u); }
    printf("\ndouble paise accumulator at 800,000 bills: %.1f  vs exact %lld  (error %.1f)\n", dt,
        exact, dt - (double)exact);
    printf("double stays exact for integers up to 2^53 = 9007199254740992; district total is far below
        it\n");
    return 0;
}
```

Output Screenshot



Output of compare.c

Analysis / Discussion

The float and paise totals agreed at 4,000 bills (Rs 3,205,507.25 in both), as expected, because every bill is a multiple of Rs 0.25 and the total is below 16,777,216. At 60,000 bills the float total was Rs 1,612.00 lower than the exact Rs 48,716,724.00, and at 800,000 bills it was Rs 1,017,911.00 higher than the exact Rs 652,891,081.00, so the float version is wrong at taluk and district scale.

The exact district total is 65,289,108,100 paise. A 32-bit int cannot hold it (INT_MAX is 2,147,483,647) and showed the wrapped value 864,598,660. long (8 bytes on this system) and long long both hold it with a large margin (LLONG_MAX is about 9.2 × 10^18). On this system short is 2, int 4, long 8 and long long 8 bytes, while the standard guarantees only 2, 2, 4 and 8. The reason for the difference is the data model: 64-bit Linux uses LP64 (long is 64 bits), whereas 64-bit Windows uses LLP64 (long is 32 bits); so long is not portable for a district total and long long should be used.

Alternative Concept and Complexity Analysis

If the district total were accumulated in a double (as paise), it was exact at 800,000 bills (65289108100.0, error 0.0), because a double stores every integer exactly up to 2^53 = 9,007,199,254,740,992; precision is lost only above that, about 1.4 × 10^5 times larger than this total. A long long loses nothing by precision but overflows at 9.2 × 10^18, and a 32-bit int overflows above 2.1 × 10^9. So overflow (integers) and precision loss (floating point) are different failures. The time per addition is O(1) for both, with a slightly slower floating-point add, and each variable needs 8 bytes; the integer is still the better choice because its result is exact by design, and for rupees as double the error appears much earlier (Task 7).

Result

The experiments matched the expectations: float is unsafe beyond the village, a 32-bit int overflows at district scale, and long long (or int64) paise is exact. The differences between this system and the C standard were explained by the LP64 data model.

TASK 11 — Recommend and Defend the Final Design

Task / Problem Statement

Recommend the final set of data types, constants and preprocessor features for the co-operative, explain how a tariff revision will be carried out, state the limitations of the design, and check whether it stays correct at district scale, using evidence from my own experiments.

Aim

To recommend and defend a final design supported by the tests and measurements of Tasks 1 to 10.

Concept / Theory

Recommended design

Item	Recommendation	Evidence
Meter reading	unsigned short (meter_t) with (meter_t)(current − previous)	Task 2: 256 units after roll-over; Task 9: all roll-over tests correct
Units, household number	unsigned int	Task 2: range 0 to 4.29 × 10^9
Money	Integer paise: long long (or int64_t) for totals, never float	Tasks 2, 7, 10: float errors of Rs 5.77 to Rs 1,017,911
Tariff constants	Named constants in one header (enum/#define); or a configuration file	Task 3: one edit changed every bill
Preprocessor features	Include guards, #ifdef DEBUG, parenthesised macros or static inline	Tasks 4 and 5
Input	fgets + sscanf with return-value checks	Tasks 6 and 9: invalid lines rejected

Tariff revision: with the compiled-in design the rate is edited once in the header (for example RATE2_PAISE 225 to 230), the program is recompiled and redistributed. With the alternative design, the rates are read from a text file when the program starts, so a revision means editing one line of the file.

Limitations: a roll-over is assumed to happen at most once and the largest plausible consumption (5000 units) is an assumption; the category does not change the tariff; long is only 32 bits on some systems, so totals must use long long; the compiled-in tariff needs a recompile; and a configuration file needs its own validation.

Procedure / Algorithm

1. Collect the evidence from Tasks 1 to 10 for every data type and constant.
2. Choose the final type for each data item and the form of the tariff constants.
3. Write a program that reads the tariff values from tariff.cfg when it starts.
4. Run it with the current tariff and with a revised tariff (RATE2 = 230) without recompiling.
5. Measure the start-up time of reading the file and the time to recompile the program.
6. Check the design at village, taluk and district scale and state its limitations.

Program / Source Code

config_tariff.c (alternative concept: tariff from a file)

```
/* Task 11 - recommended design: typed constants + tariff read from a config file at start-up */
#include <stdio.h>
#include <string.h>
#include <time.h>

typedef struct { long fixed; int s1, s2; long r1, r2, r3; } tariff_t;

static int load_tariff(const char *path, tariff_t *t)
{
    FILE *f = fopen(path, "r"); char key[32]; long v; int n = 0;
    if (!f) return -1;
```

```
    while (fscanf(f, "%31s %ld", key, &v) == 2) {
        if      (!strcmp(key, "fixed_paise")) t->fixed = v;
        else if (!strcmp(key, "slab1_units")) t->s1 = (int)v;
        else if (!strcmp(key, "slab2_units")) t->s2 = (int)v;
        else if (!strcmp(key, "rate1_paise")) t->r1 = v;
        else if (!strcmp(key, "rate2_paise")) t->r2 = v;
        else if (!strcmp(key, "rate3_paise")) t->r3 = v;
        else continue;
        n++;
    }
    fclose(f);
    return n == 6 ? 0 : -2;
}
static long bill(const tariff_t *t, int u)
{
    long p = t->fixed;
    if (u > t->s2) { p += (long)(u - t->s2) * t->r3; u = t->s2; }
    if (u > t->s1) { p += (long)(u - t->s1) * t->r2; u = t->s1; }
    return p + (long)u * t->r1;
}
int main(int argc, char **argv)
{
    tariff_t t; struct timespec a, b;
    clock_gettime(CLOCK_MONOTONIC, &a);
    int rc = load_tariff(argc > 1 ? argv[1] : "tariff.cfg", &t);
    clock_gettime(CLOCK_MONOTONIC, &b);
    if (rc) { printf("Could not load tariff (code %d)\n", rc); return 1; }
    printf("Tariff loaded in %.1f microseconds, stored in %zu bytes\n",
           (b.tv_sec - a.tv_sec) * 1e6 + (b.tv_nsec - a.tv_nsec) / 1e3, sizeof t);
    int tests[] = {0, 100, 150, 200, 350};
    for (int i = 0; i < 5; i++) { long p = bill(&t, tests[i]); printf("%3d units -> Rs %ld.%02ld\n",
        tests[i], p/100, p%100); }
    return 0;
}
```

tariff.cfg

```
fixed_paise 5000
slab1_units 100
slab2_units 200
rate1_paise 0
rate2_paise 225
rate3_paise 450
```

Output Screenshot



Output with tariff.cfg and with the revised tariff_rev.cfg (no recompilation)



Time to compile three times

Analysis / Discussion

The design remains correct at district scale: the largest district total measured was 65,289,108,100 paise, far below the long long limit of about 9.2 × 10^18, and the paise arithmetic is exact, while the float version was wrong by more than Rs 10 lakh. The recommendation is therefore based on measured evidence, not on assumptions.

Alternative Concept and Complexity Analysis

If the tariff is read from a configuration file when the program starts, the program opens the file, reads t key-value pairs (t = 6 here) and stores them in a 40-byte structure; the bill function then uses these fields instead of constants. The extra start-up time is O(t) once per run: 147.5 microseconds for the first run and 235.9 microseconds for the second in my measurement, and the extra space is 40 bytes. Compiling the program took about 32 ms (0.095 s for three compilations), and a compiled-in revision must also be recompiled, tested and redistributed to every office. With the file version, changing RATE2 from 225 to 230 changed the bill for 150 units from Rs 162.50 to Rs 165.00 and 350 units from Rs 950.00 to Rs 955.00 without any recompilation. The price is that the values are no longer checked by the compiler, so the program must check that all six values were read (it returns an error code when fewer than six are found). Given that tariffs are revised often, the configuration file is the better choice.

Result

A final design was recommended and defended with evidence: unsigned short readings, integer paise in long long, centralised tariff constants (preferably a file), and safe macros and header organisation. It stays correct at district scale.

TASK 12 — Re-evaluate and Adapt the Design (Day-4 Design Challenge)

Task / Problem Statement

The co-operative is installing smart meters that send a reading in watt-hours every 15 minutes (35,040 readings per house per year). A time-of-day tariff applies: units used between 6 PM and 10 PM cost 20% more. The tariff values must be changeable without recompiling. Re-evaluate the original design and decide whether to RETAIN, MODIFY or REDESIGN it: A. Impact Analysis, B. Evaluation of the Original Design, C. Adaptation or Redesign (with working code), D. Evidence-Based Defence.

Aim

To re-evaluate the original billing design against the changed requirements and to implement and defend a revised design.

Concept / Theory

A. Impact Analysis

- What changed: readings arrive in watt-hours every 15 minutes (96 a day, about 2,880 a month, 35,040 a year) instead of one reading a month; the bill now depends on the time of day (6 PM to 10 PM costs 20% more); and the tariff must be changeable without recompiling.
- Data types no longer sufficient: a 16-bit register holds only 65,535 Wh, which a 20 kW connection fills in 13.1 readings (3.3 hours), so a 32-bit register is needed; yearly totals in Wh per house reach 175,200,000 and the district total reaches 1.4 × 10^14 Wh, which does not fit in 32 bits and needs 64 bits.
- Constants that can no longer be #define: the slab limits, the rates, the fixed charge, and the peak hours and surcharge, because they must be read at run time from a file.

B. Evaluation of the Original Design

Largest values that can now occur (assuming a 20 kW connection, 5000 Wh per reading): per house per year 35,040 × 5,000 = 175,200,000 Wh (fits int, but only because it is one house); district per year 1.4016 × 10^14 Wh (UINT32_MAX = 4,294,967,295, so 32 bits is not enough); the largest yearly bill of one house is 94,608,000 paise (fits int, but a district sum of 7.57 × 10^13 paise does not); the scaled accumulator of one house-year is 9.46 × 10^12, which fits int64. Evaluation: the unsigned short meter_t is no longer appropriate; long for money is not portable (32 bits on Windows) and must become int64_t; integer paise, include guards, the DEBUG mode and the slab method remain appropriate; #define tariff constants are no longer appropriate; int units in kWh must become a 64-bit count of Wh.

C. Adaptation or Redesign

Decision: MODIFY (keep the integer-paise slab design, change the types and the way the constants are supplied). The redesign: (1) reg_t is uint32_t with consumption = (reg_t)(cur − prev); (2) money is int64_t paise; (3) every 15-minute reading is priced when it arrives: the cumulative Wh of the month decides the slab, a reading that crosses a slab boundary is split, and a reading in the peak hours (6 PM to 10 PM) is multiplied by (100 + 20)%; (4) the charges are accumulated in integer units of paise / 100,000 (Wh × paise per kWh × percentage) and rounded to paise once, half up, at the end of the bill; (5) all tariff values are read from tariff12.cfg when the program starts.

Procedure / Algorithm

1. Read the tariff (fixed charge, slab limits, rates, peak hours, peak percentage) from tariff12.cfg.
2. For every 15-minute reading compute the Wh used with 32-bit wrap-around subtraction.
3. Find the slab from the cumulative Wh of the month and split a reading that crosses a slab limit.
4. Multiply the charge of a reading in the peak hours (18:00 to 22:00) by 120 percent.
5. Accumulate in 64-bit integer units and round once to paise at the end of the bill.
6. Run the built-in self test, a 30-day simulation, a revised tariff file, and the largest-value calculation.

Program / Source Code

smart.c

```
/* Task 12 - Smart-meter redesign: 15-minute Wh readings, peak-hour rule, tariff read at run time */
#include <stdio.h>
#include <string.h>
#include <stdint.h>
#include <inttypes.h>
#include <limits.h>

typedef uint32_t reg_t;            /* meter register in watt-hours (32-bit, wraps at 2^32) */
typedef int64_t  paise_t;          /* money: integer paise, always 64-bit                   */

typedef struct {
    paise_t fixed; long long slab1_wh, slab2_wh; long rate1, rate2, rate3;   /* paise per kWh */
    int peak_start, peak_end, peak_pct;
} tariff_t;

static int load_tariff(const char *path, tariff_t *t)
{
    FILE *f = fopen(path, "r"); char key[40]; long v; int n = 0;
    if (!f) return -1;
    while (fscanf(f, "%39s %ld", key, &v) == 2) {
        if      (!strcmp(key, "fixed_paise"))           t->fixed = v;
        else if (!strcmp(key, "slab1_kwh"))             t->slab1_wh = v * 1000LL;
        else if (!strcmp(key, "slab2_kwh"))             t->slab2_wh = v * 1000LL;
        else if (!strcmp(key, "rate1_paise_per_kwh"))   t->rate1 = v;
        else if (!strcmp(key, "rate2_paise_per_kwh"))   t->rate2 = v;
        else if (!strcmp(key, "rate3_paise_per_kwh"))   t->rate3 = v;
        else if (!strcmp(key, "peak_start_hour"))       t->peak_start = (int)v;
        else if (!strcmp(key, "peak_end_hour"))         t->peak_end = (int)v;
        else if (!strcmp(key, "peak_surcharge_percent")) t->peak_pct = (int)v;
        else continue;
        n++;
    }
    fclose(f);
    return n == 9 ? 0 : -2;
}

static int is_peak(const tariff_t *t, int hour) { return hour >= t->peak_start && hour < t->peak_end; }

/* energy charge of one interval, in units of (paise / 100000); splits across slab boundaries */
static int64_t price_interval(const tariff_t *t, long long *cum_wh, unsigned wh, int hour)
{
    int64_t scaled = 0; long long left = wh;
    int mult = 100 + (is_peak(t, hour) ? t->peak_pct : 0);
    while (left > 0) {
        long rate; long long room;
        if      (*cum_wh < t->slab1_wh) { rate = t->rate1; room = t->slab1_wh - *cum_wh; }
        else if (*cum_wh < t->slab2_wh) { rate = t->rate2; room = t->slab2_wh - *cum_wh; }
        else                            { rate = t->rate3; room = left; }
        long long take = left < room ? left : room;
        scaled += (int64_t)take * rate * mult;
        *cum_wh += take; left -= take;
    }
    return scaled;
}
static paise_t round_paise(int64_t scaled) { return (scaled + 50000) / 100000; }   /* rounded once, half
     up */
static reg_t   consumption(reg_t prev, reg_t cur) { return (reg_t)(cur - prev); }

static int selftest(const tariff_t *t)
{
    struct { const char *name; long long cum; unsigned wh; int hour; paise_t want; } c[] = {
        {"1 kWh, free slab, off-peak",       0,      1000, 12,   0},
        {"1 kWh, slab 2, off-peak (12:00)",  100000, 1000, 12, 225},
        {"1 kWh, slab 2, peak (19:00)",      100000, 1000, 19, 270},
        {"1 kWh, slab 3, peak (21:45)",      200000, 1000, 21, 540},
        {"1 kWh, slab 2, hour 22 (not peak)",100000, 1000, 22, 225},
        {"1 kWh, slab 2, hour 17 (not peak)",100000, 1000, 17, 225},
        {"1 kWh spanning slab 1/2 boundary", 99500,  1000, 12, 113},
    };
    int fails = 0;
    for (int i = 0; i < 7; i++) {
        long long cum = c[i].cum; paise_t got = round_paise(price_interval(t, &cum, c[i].wh,
```

```
            c[i].hour));
        printf("%-36s expected %4" PRId64 " paise, got %4" PRId64 "  %s\n", c[i].name, c[i].want, got,
            got == c[i].want ? "PASS" : "FAIL");
        fails += got != c[i].want;
    }
    reg_t u = consumption(4294967000u, 500u);
    printf("%-36s expected  796 Wh,   got %4u  %s\n", "32-bit register roll-over", (unsigned)u, u == 796
         ? "PASS" : "FAIL");
    fails += u != 796;
    return fails;
}

static void limits(void)
{
    long double max_wh = 5000.0L;                           /* 20 kW connection x 0.25 h */
    long double per_house_year = 35040.0L * max_wh;
    long double district_year  = per_house_year * 800000.0L;
    long double max_bill = (per_house_year / 1000.0L) * 450.0L * 1.20L;   /* paise */
    printf("Assumed worst case: 20 kW supply -> %.0Lf Wh per 15-minute reading\n", max_wh);
    printf("16-bit register holds 65535 Wh -> full in %.1Lf readings (%.1Lf hours)\n", 65535.0L /
        max_wh, 65535.0L / max_wh * 0.25L);
    printf("Readings per house per year        : 35040\n");
    printf("Wh per house per year   (max)      : %.0Lf   (INT_MAX = %d)\n", per_house_year, INT_MAX);
    printf("Wh per district per year (800,000) : %.4Le  (UINT32_MAX = %" PRIu32 ")\n", district_year,
        UINT32_MAX);
    printf("Largest yearly bill, one house     : %.0Lf paise  (INT_MAX = %d)\n", max_bill, INT_MAX);
    printf("District yearly billing (paise)    : %.4Le  (INT64_MAX = %.4Le)\n", max_bill * 800000.0L,
        (long double)INT64_MAX);
    printf("Scaled accumulator per house-year  : %.4Le  (fits int64: %s)\n", per_house_year * 450.0L *
        120.0L,
           per_house_year * 450.0L * 120.0L < (long double)INT64_MAX ? "yes" : "NO");
    printf("Raw 4-byte readings per house-year : %.0Lf bytes; district: %.1Lf GB\n", 35040.0L * 4,
        35040.0L * 4 * 800000.0L / 1e9L);
}

int main(int argc, char **argv)
{
    const char *cfg = "tariff12.cfg"; int mode = 0;
    for (int i = 1; i < argc; i++) {
        if (!strcmp(argv[i], "--selftest")) mode = 1;
        else if (!strcmp(argv[i], "--limits")) mode = 2;
        else cfg = argv[i];
    }
    if (mode == 2) { limits(); return 0; }
    tariff_t t;
    if (load_tariff(cfg, &t)) { printf("Cannot load tariff file %s\n", cfg); return 1; }
    printf("Tariff file %s: slab rates %ld/%ld/%ld paise per kWh, peak %02d:00-%02d:00 +%d%%\n", cfg,
        t.rate1, t.rate2, t.rate3, t.peak_start, t.peak_end, t.peak_pct);
    if (mode == 1) { int f = selftest(&t); printf("%s\n", f ? "SOME TESTS FAILED" : "ALL TESTS PASSED");
         return f != 0; }

    /* one house, 30 days x 96 intervals, generated deterministically */
    uint32_t seed = 7; long long cum = 0, peak_wh = 0; int64_t scaled = 0; reg_t reg = 4294960000u,
        first = reg;
    for (int k = 0; k < 30 * 96; k++) {
        int hour = (k % 96) / 4;
        seed = seed * 1103515245u + 12345u; unsigned r = (seed >> 16) & 0x7fff;
        unsigned wh = 40 + r % 80 + (is_peak(&t, hour) ? 100 + (r / 80) % 100 : 0);
        reg_t prev = reg; reg = (reg_t)(reg + wh);                    /* register wraps at 2^32 */
        wh = consumption(prev, reg);
        if (is_peak(&t, hour)) peak_wh += wh;
        scaled += price_interval(&t, &cum, wh, hour);
    }
    paise_t energy = round_paise(scaled), total = t.fixed + energy;
    printf("Readings processed      : %d (30 days x 96)\n", 30 * 96);
    printf("Register start / end    : %" PRIu32 " -> %" PRIu32 " Wh (wrapped past 2^32: %s)\n", first,
        reg, reg < first ? "yes" : "no");
    printf("Total consumption       : %lld Wh = %.3f kWh (peak: %lld Wh)\n", cum, cum / 1000.0,
        peak_wh);
    printf("Energy charge           : Rs %" PRId64 ".%02" PRId64 "\n", energy / 100, energy % 100);
    printf("Fixed charge            : Rs %" PRId64 ".%02" PRId64 "\n", t.fixed / 100, t.fixed % 100);
    printf("Total bill              : Rs %" PRId64 ".%02" PRId64 "\n", total / 100, total % 100);
    return 0;
}
```

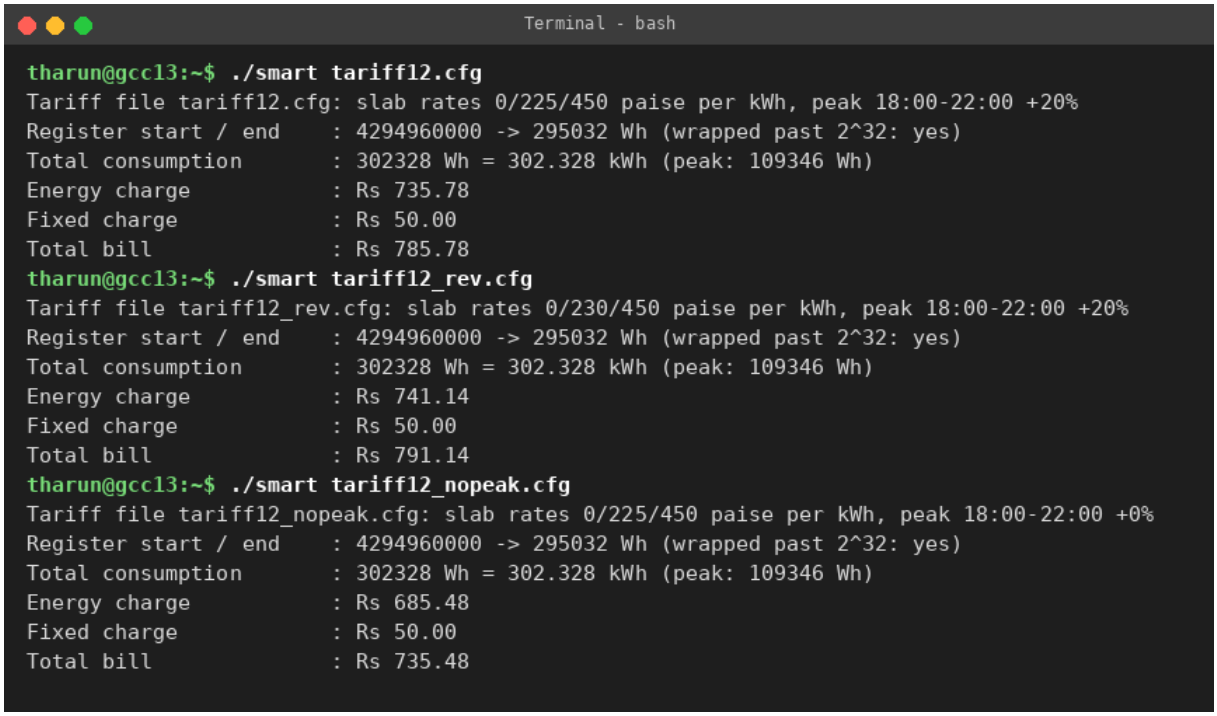
tariff12.cfg

```
fixed_paise 5000
slab1_kwh 100
slab2_kwh 200
rate1_paise_per_kwh 0
rate2_paise_per_kwh 225
rate3_paise_per_kwh 450
peak_start_hour 18
peak_end_hour 22
peak_surcharge_percent 20
```

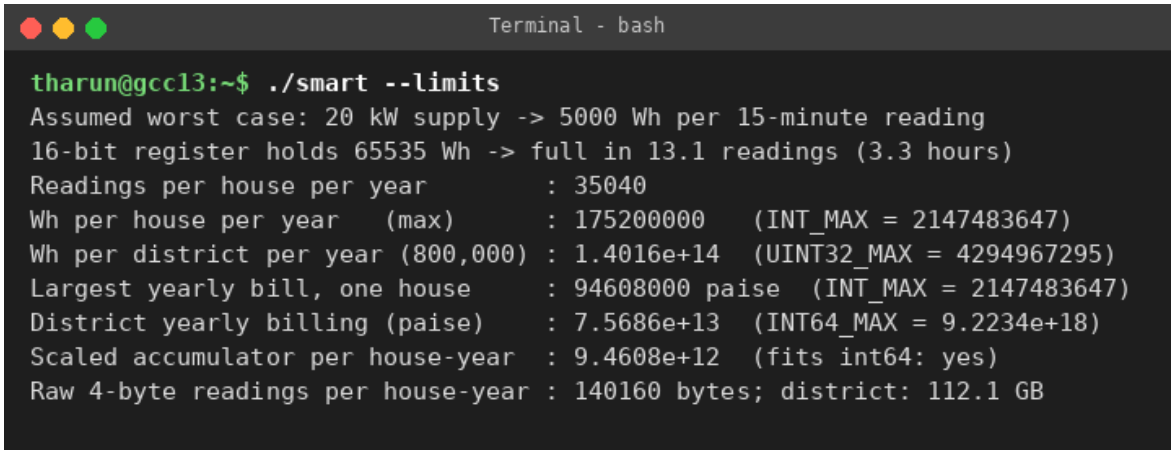
Output Screenshot



Self test of the peak-hour rule, slab boundary and 32-bit roll-over



30-day bill: current tariff, revised tariff (RATE2 = 230) and no peak rule



Largest values that can now occur

Analysis / Discussion

D. Evidence-Based Defence

All 8 self tests passed: 1 kWh in slab 2 costs 225 paise off-peak and 270 paise at 19:00 (exactly 20% more); in slab 3 at 21:45 it costs 540 paise; at 22:00 and 17:00 (outside the peak) it costs 225 paise; a reading crossing the free-slab boundary (500 Wh free and 500 Wh at Rs 2.25) gives 112.5 paise, rounded once to 113; and the 32-bit register roll-over from 4,294,967,000 to 500 gives 796 Wh. In the 30-day simulation (2,880 readings, 302.328 kWh, of which 109.346 kWh in the peak hours) the register wrapped past 2^32 and the bill was Rs 785.78. An independent Python calculation with exact fractions gave the same bill. Without the peak rule the same consumption costs Rs 735.48, so the peak rule adds Rs 50.30. Changing rate2 to 230 in the file gave Rs 791.14 without recompiling.

The decision to modify rather than retain is justified by the numbers: the 16-bit register would be full after 3.3 hours, the district totals need more than 32 bits (1.4 × 10^14 Wh and 7.57 × 10^13 paise), and the tariff has to be edited without a compiler. The decision to modify rather than redesign is justified because the integer-paise slab design, the rules of Task 1, the input checks and the header and DEBUG organisation all remained correct. Trade-offs: reading the file adds a start-up cost of about 0.2 ms and requires validating the file (the program reports an error unless all 9 values are found); the interval loop costs O(r) per house for r readings (2,880 a month) instead of one calculation; and storing every reading takes about 140 KB per house-year (112 GB for the district), so the readings should be priced and discarded when they arrive.

Result

The original design was modified: wider unsigned and 64-bit types, tariff values read at run time, and a peak-hour rule were implemented and verified by 8 tests, a month simulation and the largest-value calculation, and defended with measured evidence.

Test Cases / Validation Table

Record different inputs, expected results, actual results, and whether each test passed.

Test No.	Input / Scenario	Expected Output	Actual Output	Status
1	Task 9: 50 units (house 101)	Rs 50.00	Rs 50.00	PASS
2	Boundary: exactly 100 units	Rs 50.00	Rs 50.00	PASS
3	101 units (slab 2 starts)	Rs 52.25	Rs 52.25	PASS
4	Boundary: exactly 200 units	Rs 275.00	Rs 275.00	PASS
5	201 units (slab 3 starts)	Rs 279.50	Rs 279.50	PASS
6	Zero units (500 to 500)	Rs 50.00	Rs 50.00	PASS
7	Roll-over 65400 to 120	256 units, Rs 527.00	256 units, Rs 527.00	PASS
8	Roll-over 65535 to 0	1 unit, Rs 50.00	1 unit, Rs 50.00	PASS
9	Invalid: letters (abc)	Rejected	Rejected	PASS
10	Invalid: negative reading (-5)	Rejected	Rejected	PASS
11	Invalid: 5000 to 4000, no roll-over	Rejected	Rejected	PASS
12	Invalid: reading 70000 (above 65535)	Rejected	Rejected	PASS
13	5 hand-calculated bills (150, 200, 350, 0, 120 units)	Rs 1532.50	Rs 1532.50	PASS
14	Task 3: RATE2_PAISE = 230, 150 units	Rs 165.00	Rs 165.00	PASS
15	Task 10: 800,000 bills in long long (paise)	65289108100	65289108100	PASS
16	Task 12: 1 kWh in slab 2 at 19:00 (peak)	270 paise	270 paise	PASS
17	Task 12: 32-bit roll-over 4294967000 to 500	796 Wh	796 Wh	PASS
18	Task 12: 30-day bill, tariff12.cfg	Rs 785.78	Rs 785.78	PASS

Final Submission Checklist

☑️ Student and course details completed

☑️ Task statement, aim, theory, and algorithm included

☑️ Program code is readable and runs successfully

☑️ Output screenshot included

☐ Analysis and result written in own words

☑️ Test cases completed where applicable

☐ GitHub repository link added if required